----------VERİNİN YOLCULUĞU----------



Diyelim ki bir E-ticaret şirketinde çalışıyorsunuz. Bu şirket Türkiyenin farklı şehirlerine farklı kategorilerde ürünler satan bir şirket. Yöneticiniz gelip size “Biz bu şirkette ne kadar satış yapıyoruz, neyi nerde en çok satıyoruz görmek istiyorum” dedi. Bizim bu isteği yerine getirebilmemiz için önce verileri tutmamız lazım.

Verileri Nerede Tutuyoruz?

Gerçek hayatta, özellikle yüksek hacimli e-ticaret sistemlerinde önceliğin başında hız gelir. Bir müşteri "Satın Al" butonuna bastığı anda, sistemin öncelikli görevi bu isteği en hızlı ve en güvenilir şekilde kaydetmekti. Çünkü o anda sistemin yavaşlaması ya da hata vermesi doğrudan satış kaybı demektir. Bu nedenle sipariş anında çalışan sistemler (OLTP-online Transaction Processing sistemleri), veriyi kaydederken karmaşık doğrulama, temizleme ya da standartlaştırma adımlarından kaçınır; sadece "veri kayboldu mu, kaydedildi mi" sorusuna odaklanır.

Bu yüzden birçok şirket, sipariş verisini ilk aşamada ağır kısıtlamaları olan klasik ilişkisel veritabanlarına değil, yazma işlemine optimize edilmiş sistemlere yazar. Bunlar bazen HBase, Cassandra gibi NoSQL veritabanları, bazen de bir "data lake" içindeki ham dosya depolarıdır (örneğin Amazon S3, Hadoop HDFS). Bu sistemlerin ortak özelliği, "schema-on-write" değil "schema-on-read" mantığıyla çalışmasıdır. Yani veri yazılırken sıkı bir şema (format, veri tipi, zorunlu alan) kontrolü yapılmaz; veri neyse öyle kaydedilir, hangi formatta anlam kazanacağına ise veri daha sonra okunurken karar verilir.

Bu yaklaşımın avantajı açıktır: sipariş akışı hiçbir zaman bir veri doğrulama hatası yüzünden durmaz. Ancak bunun bir bedeli vardır: aylar içinde bu sistemlerde, aynı bilginin onlarca farklı şekilde yazıldığı, bazı alanların boş bırakıldığı, bazı sayısal değerlerin metin olarak girildiği devasa bir ham veri yığını birikir. Bu veri, o anda kimseyi rahatsız etmez. Çünkü sipariş süreci sorunsuz işlemektedir. Sorun, bu veri üzerinden bir analiz yapılmak istendiği anda ortaya çıkar: artık kimse "şehre göre" ya da "kategoriye göre" güvenilir bir toplam alamaz, çünkü veri hiçbir zaman bu soruları cevaplayacak şekilde düzenlenmemiştir.

Bir sipariş verisi geldi ve bu bahsettiğimiz Data Lake'e kaydettik diyelim.

order_id	order_date	customer_name	customer_email	customer_city	customer_segment	product_name
ORD-10378	31/08/2025	Savak Tevetoğlu	sakaryacannur@example.com	Ankara	Kurumsal	Mutfak Robotu

product_category	quantity	unit_price	total_amount	payment_method	shipping_city	order_status
Ev & Yaşam	4	2644.22	17691.6	Kapıda Ödeme	Kayseri	Tamamlandı

Ardından başka bir sipariş daha geldi ve bunun da verisi kaydedildi diyelim.

order_id	order_date	customer_name	customer_email	customer_city	customer_segment	product_name
ORD-10100	01.08.2025	Feryas Gilman Demirel Alemdar	eraslansevketfeza@example.com	ADANA	Kurumsal	Kablosuz Kulaklık

product_category	quantity	unit_price	total_amount	payment_method	shipping_city	order_status
Elektronik	3	888.43	2665.3	Dijital Cüzdan	Adana	Tamamlandı

Görüldüğü üzere girilen veriler farklı formatta. Örneğin customer_city bir örnekte sadece ilk harfi büyükken başka bir örnekte tamamen büyük yazılmış. order_date kısmında bir kullanıcı yılı en başa yazarken bir kullanıcı en sona yazmış ve farklı ayırıcılar kullanmışlar. Ancak bu verilerin toplanmasında hız önemli olduğu için bu tutarsızlıklar dikkat edilmez.
Ardından diyelim ki 2025 yılı boyunca bir yıllık veri toplandı ve 510 satırlık veri kaydedildi.
Artık bu verileri analiz etmek istiyoruz. ancak analizi yapmamız için verileri düzenlemeliyiz.

Examine preview data

Rows of step: ecommerce_raw_data (510 rows)

	order_id	order_date	customer_name	customer_email	customer_city	customer_segment	product_name	produ
1	ORD-10382	2025-09-01	Refet Çamurcuoğlu	gtevetoglu@example.org	Mersin	Bireysel	Cilt Bakım Seti	Kozme
2	ORD-10480	2025-09-17	Badegül Şama	mucevherakca@example.net	Eskişehir	Bireysel	Ofis Sandalyesi	Ev & Ya
3	ORD-10076	2025-12-06	Çev. Sevla Çorlu Gül	lduran@example.com	Mersin	Kurumsal	Cilt Bakım Seti	Kozme
4	ORD-10417	2025-03-09	Tayyibe Mansız	odurdu@example.org	Bursa	Bireysel	Ofis Sandalyesi	Ev & Ya
5	ORD-10420	22.12.2025	Öğr. Güverçin Manço Duran	vyuksel@example.net	Eskişehir	Bireysel	Çocuk Oyuncak Seti	Anne &
6	ORD-10353	2025-08-25	Vâlâ Demir	valaakgunduz@example.com	İstanbul	Bireysel	Ofis Sandalyesi	Ev & Ya
7	ORD-10024	2025-02-20	Misra Şahdiye Şener Akçay	akarsoykan@example.org	İstanbul	Kurumsal	Roman Seti (5 Kitap)	Kitap &
8	ORD-10235	2025-07-05	Yrd. Doç. Kutgün Nurgil Durmuş	biltas90@example.org	ANKARA	Bireysel	Akıllı Saat	Elektro
9	ORD-10190	2025-08-17	Abdülkerim Bilgin	kopan85@example.net	Bursa	Kurumsal	Cilt Bakım Seti	Kozme
10	ORD-10384	2025-03-06	Av. Sayin Uğurkan Durdu	erbaybilir@example.net	İstanbul	Kurumsal	Laptop Standı	ELEKTF
11	ORD-10428	2025-02-28	Yurtseven Soylu Korutürk	ilklimakisakurek@example.net	Konya	Kurumsal	Güneş Gözlüğü	Aksesu
12	ORD-10038	2025-12-20	Serezli Yılmaz	sanli85@example.com	İZMIR	Bireysel	Masaüstü Bilgisayar	Elektro
13	ORD-10460	2025-11-29	Ilgı Eraslan	arcan52@example.com	İstanbul	Bireysel	Masaüstü Bilgisayar	elektro
14	ORD-10451	2025-07-03	Şenkal Talayer Zengin Bilge	camurcuoglusahil@example.net	Gaziantep	Bireysel	Akıllı Telefon	Elektro
15	ORD-10172	2025-05-27	Arş. Gör. Gülcegün Meryeme Şensoy Akdeniz	bilirihsan@example.net	Gaziantep	Bireysel	Cilt Bakım Seti	Kozme
16	ORD-10442	2025-11-21	Elöve Ergül	ferhan92@example.com	Eskişehir	Bireysel	Güneş Gözlüğü	AKSES
17	ORD-10044	2025-06-19	Oganer İnönü	ongay22@example.net	İstanbul	Bireysel	Kadın Elbise	Giyim
18	ORD-10085	2025-04-19	Çaltı Mishat Tarhan Yüksel	rarsoy@example.com	Antalya	Bireysel	Yoga Matı	Spor
19	ORD-10219	2025-10-06	Dr. Gülselin Emal Yılmaz	durmusensari@example.net	Konya	Kurumsal	Masaüstü Bilgisayar	Elektro

Veri setindeki bazı başka düzenleme gerektiren durumlar:



product_category
Giyim
Ev & Yaşam
Spor
Elektronik
Kitap & Kırtasiye
Aksesuar
KOZMETIK
Spor

quantity
3
5
5
5
4
4
1
5
iki
6

customer_email
dirican90@example.com
bulker@example.com
<null>
buyrukhanyuksel_at_example.net
yukselruknettin@example.com

quantity
3
5
5
5
4
4
1
5
iki
6

Bu verileri düzenleyip analiz etmemiz lazım. Ama bu düzenlemeleri verileri yüklediğimiz data lake'de yapamayız. Çünkü data lake'in temel felsefesi ham veriyi olduğu gibi, değiştirmeden saklamaktır. Bu prensibe genelde "immutability" (değişmezlik) denir ve iki önemli nedeni vardır:

Birincisi, kaynak veri bir kanıt/arşiv niteliği taşır. Eğer ham veriyi olduğu yerde düzenlersek, ileride "bu sipariş gerçekte ne zaman, hangi bilgilerle geldi?" sorusuna cevap veremez hale geliriz. orijinal kayıt kaybolur. Bir hata ya da tartışmalı durum olduğunda geri dönüp bakılacak "gerçek" kayıt, hep bu ham hali olmalıdır.

İkincisi, teknik olarak bu sistemler buna uygun tasarlanmamıştır. Data lake'ler (S3, HDFS gibi) satır satır düzenleme (update) işlemine göre değil, büyük hacimli veriyi ucuza ve hızlıca depolamaya göre optimize edilmiştir. Milyonlarca satır arasından "quantity alanı 'iki' yazan satırları bul ve 2'ye çevir" gibi bir işlem yapmak, bu sistemlerin doğasına aykırıdır ve pratik değildir.

Bu yüzden veriyi düzenlemek için, onu başka bir ortama çıkarmamız (extract etmemiz) gerekir. Orijinal ham veriye dokunmadan, bir kopyasını alıp, üzerinde çalışabileceğimiz bir işlem ortamına taşımamız gerekir. İşte tam bu noktada devreye ETL sürecinin ilk adımı olan Extract girer.

EXTRACT:

Gerçek hayatta Extract, ham verinin bulunduğu kaynaktan (data lake, NoSQL veritabanı, işletim sistemi kayıtları vb.) alınıp, üzerinde işlem yapılacak ayrı bir ortama kopyalanması işlemidir. Buradaki kritik nokta şudur: extract, veriyi taşımaz, çoğaltır. kaynaktaki orijinal veri olduğu gibi kalır, işlenecek veri ise ayrı bir kopya üzerinden ilerler. Bu sayede bir önceki bölümde bahsettiğimiz "ham veri bir kanıt niteliği taşır" prensibi korunmuş olur. Gerçek dünyada extract işlemi genelde iki şekilde yapılır:

Tam çıkarma (full extraction): Kaynaktaki verinin tamamı, her seferinde baştan alınır. Küçük ölçekli veri setlerinde ya da ilk kurulumda tercih edilir.

Artımlı çıkarma (incremental extraction): Sadece bir önceki çalıştırmadan bu yana eklenen/değişen veri alınır (genelde bir tarih veya sıra numarası referans alınarak). Büyük ölçekli sistemlerde, her seferinde milyonlarca satırı yeniden taşımamak için tercih edilir.

Bu işlemi otomatikleştirmek için şirketler genelde özel araçlar kullanır. örneğin Apache Sqoop (ilişkisel veritabanlarından Hadoop'a veri taşımak için), Kafka Connect (akan/canlı veriyi taşımak için) ya da bizim projemizde de kullandığımız Pentaho Data Integration gibi genel amaçlı ETL araçları.
Bizim projemizde ise bu adım şöyle işledi: yıl boyunca biriken 510 satırlık ham veri (ecommerce_raw_data.csv), kaynağında (data lake benzetmemizdeki dosya deposunda) hiç değiştirilmeden, olduğu gibi VM2 (pentaho-server) üzerine taşındı. Burada iki ayrı sanal makine kullanmamızın nedeni de tam olarak bu ayrımı yansıtmaktı: VM1 (db-server), verinin nihai olarak yaşayacağı, temiz ve güvenilir veri ambarını temsil ediyordu; VM2 (pentaho-server) ise ham verinin işleneceği, üzerinde deneme-yanılma yapılabilecek "çalışma alanı" rolündeydi.

VM2 üzerinde Pentaho Data Integration'ın Spoon arayüzünde, bir CSV File Input adımı ile bu dosya okunarak transformation akışına dahil edildi. Bu, verinin kaynak sistemden çıkıp ilk kez "işlenebilir" bir ortama girdiği andı. Ancak henüz hiçbir temizlik yapılmamıştı; veri hâlâ, bir önceki bölümde listelediğimiz tüm sorunlarıyla birlikteydi. Extract aşamasının amacı veriyi düzeltmek değil, sadece güvenli ve eksiksiz şekilde bir sonraki aşamaya (Transform) taşımaktı.

Neden İki Ayrı Sunucu Kullandık?

Gerçek hayatta büyük ölçekli veri sistemlerinde, veritabanı sunucusu ile ETL/işlem sunucusu neredeyse hiçbir zaman aynı makine üzerinde çalıştırılmaz. Bunun birkaç somut nedeni vardır:
Kaynak ayrımı (resource isolation): ETL işlemleri özellikle büyük veri setlerinde temizlik, dönüştürme ve birleştirme adımları yoğun CPU ve bellek tüketir. Eğer bu işlemler veritabanıyla aynı makinede çalışırsa, ETL sürecinin yoğunlaştığı anlarda veritabanının performansı düşer; canlı sistemde sorgu çeken diğer kullanıcılar (örneğin dashboard'a bakan bir yönetici) bundan doğrudan etkilenir.
Ölçeklenebilirlik: İhtiyaç arttığında, sadece ETL sunucusunu güçlendirmek (veya birden fazla ETL sunucusu eklemek) mümkün olur, veritabanına dokunmaya gerek kalmaz ya da tam tersi.

Bizim projemizde bu ayrımı, gerçek bir üretim ortamındaki gibi fiziksel/network düzeyinde iki farklı makineyle (VM1 ve VM2) kurduk: VM1 (db-server) yalnızca veriyi barındırma ve sorgulanabilir hale getirme görevini üstlenirken, VM2 (pentaho-server) yalnızca ham veriyi işleme görevini üstlendi. İki makine de Bridged Adapter ile aynı ağda, birbirini IP üzerinden görebilecek şekilde yapılandırıldı tıpkı gerçek bir şirket ağında farklı sunucuların birbirleriyle konuşması gibi.

Neden Docker Kullandık?

Gerçek hayatta şirketler, PostgreSQL gibi veritabanı sistemlerini doğrudan işletim sistemine kurmak yerine genellikle container (konteyner) teknolojisiyle çalıştırır. bunun en yaygın aracı Docker'dır. Bunun nedenleri şunlardır:

Tutarlılık: Docker ile çalışan bir veritabanı, "benim makinemde çalışıyordu" sorununu ortadan kaldırır. çünkü container, içindeki yazılımın hangi sürümde, hangi ayarlarla çalıştığını bir "image" içine sabitler. Bu image, geliştirici bilgisayarında da, test sunucusunda da, canlı sunucuda da birebir aynı şekilde çalışır.

Hızlı kurulum ve temizlik: PostgreSQL'i sıfırdan işletim sistemine kurup yapılandırmak yerine, docker-compose up komutuyla saniyeler içinde hazır bir veritabanı ayağa kaldırılabilir; işiniz bittiğinde de sistemde kalıntı bırakmadan kaldırılabilir.

İzolasyon: Container içindeki yazılım, kendi bağımlılıklarıyla (kütüphane sürümleri vb.) izole bir şekilde çalışır; bu, ana işletim sistemini veya üzerinde çalışan başka yazılımları etkilemez.

Bizim projemizde VM1 üzerine Docker Engine kurularak, PostgreSQL 16 ve pgAdmin4 birer container olarak docker-compose ile ayağa kaldırıldı. VM2 tarafında ise Pentaho'yu Docker içinde çalıştırmak, grafik arayüzünün (Spoon) container içinden ana makineye yönlendirilmesini gerektireceğinden (X11 forwarding) gereksiz bir karmaşıklık yaratacaktı; bu nedenle Pentaho doğrudan VM2'nin işletim sistemine kuruldu. Bu, "her yerde Docker kullan" kuralının değil, her aracı, o aracın ihtiyacına göre en uygun şekilde kurma mantığının bir örneğiydi. gerçek projelerde de araçlar genelde bu şekilde, ihtiyaca göre seçilir.

Neden Pentaho Kullandık?

Gerçek hayatta ETL işlemleri için şirketler iki yoldan birini seçer. Birincisi, kodla (Python, SQL script'leri, özel yazılım) sıfırdan bir ETL süreci yazmaktır. İkincisi ise, bu işi görsel bir arayüz üzerinden, hazır bileşenleri sürükle bırak mantığıyla birleştirerek yapan ETL araçlarını kullanmaktır. Pentaho Data Integration, Talend, Informatica, Microsoft SSIS gibi araçlar bu ikinci gruba girer. Görsel ETL araçlarının tercih edilmesinin birkaç nedeni vardır:

Geliştirme hızı: Her adımı (dosya okuma, alan temizleme, arama/lookup, veritabanına yazma vb.) sıfırdan kod yazmak yerine, hazır bileşenleri sürükleyip parametrelerini doldurmak, süreci belirgin şekilde hızlandırır.
Görünürlük ve bakım kolaylığı: Bir transformation akışı, kod satırları yerine kutular ve oklarla temsil edildiği için, sürece sonradan dahil olan biri (ya da aylar sonra aynı kişi) akışı kod okumadan, sadece şemaya bakarak anlayabilir.
Hata ayıklama: Pentaho gibi araçlar, her adımın kaç satır aldığını, kaç satır işlediğini ve kaç satır hata verdiğini (I, O, R, W, U, E sayaçları gibi) canlı olarak gösterir. Bu, bir kod tabanında print/log satırları eklemeden çok daha hızlı bir gözlem imkânı sunar.

Pentaho özelinde ise, açık kaynaklı sürümünün (PDI Community Edition) ücretsiz olması, üniversite ve öğrenme projeleri için erişilebilir bir seçenek olmasını sağlar; kurumsal araçlar (Informatica, SSIS gibi) genelde lisans maliyeti gerektirir.

Bizim projemizde bu nedenlerle Pentaho Data Integration (PDI-CE 9.4) tercih edildi. 29 adımlık transformation akışı, VM2 üzerinde Spoon arayüzünde görsel olarak kuruldu; her adımın işlediği satır sayısı canlı olarak izlenebildi ve bu proje boyunca karşılaşılan hataların kök nedenini bulmak, bu görünürlük sayesinde mümkün oldu.

Şemayı Nasıl Oluşturduk?

Extract aşamasında ham veriyi VM2'ye taşımış olsak da, bu veriyi henüz hiçbir yere yükleyemezdik; çünkü kendisini kabul edecek bir yapı, yani hedef veritabanında tanımlanmış tablolar henüz yoktu. Gerçek hayatta da ETL süreçlerinde veri akıtılmadan önce, hedef veritabanının şeması önceden tasarlanır ve kurulur. ETL aracı, kendi başına tablo yapısına karar vermez; sadece kendisine önceden tanımlanmış bir yapıya veri akıtır. Bu nedenle, Transformation aşamasına geçmeden önce VM1 üzerindeki PostgreSQL veritabanında, ham verinin analiz edilebilir hale geldiğinde yerleşeceği yıldız şeması pgAdmin üzerinden SQL ile kuruldu. Bu şema, dört boyut (dimension) tablosu ile bir olgu (fact) tablosundan oluşuyor:

```
DROP TABLE IF EXISTS fact_orders CASCADE;
DROP TABLE IF EXISTS dim_customer CASCADE;
DROP TABLE IF EXISTS dim_product CASCADE;
DROP TABLE IF EXISTS dim_date CASCADE;
DROP TABLE IF EXISTS dim_payment CASCADE;

CREATE TABLE dim_customer (
    customer_id SERIAL PRIMARY KEY,
    customer_name VARCHAR(150),
    customer_email VARCHAR(150),
    customer_city VARCHAR(100),
    customer_segment VARCHAR(50)
);

CREATE TABLE dim_product (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(150),
    product_category VARCHAR(100)
);

CREATE TABLE dim_date (
    date_id SERIAL PRIMARY KEY,
    full_date DATE UNIQUE,
    day INT,
    month INT,
    year INT,
    quarter INT
);

CREATE TABLE dim_payment (
    payment_id SERIAL PRIMARY KEY,
    payment_method VARCHAR(100) UNIQUE
);

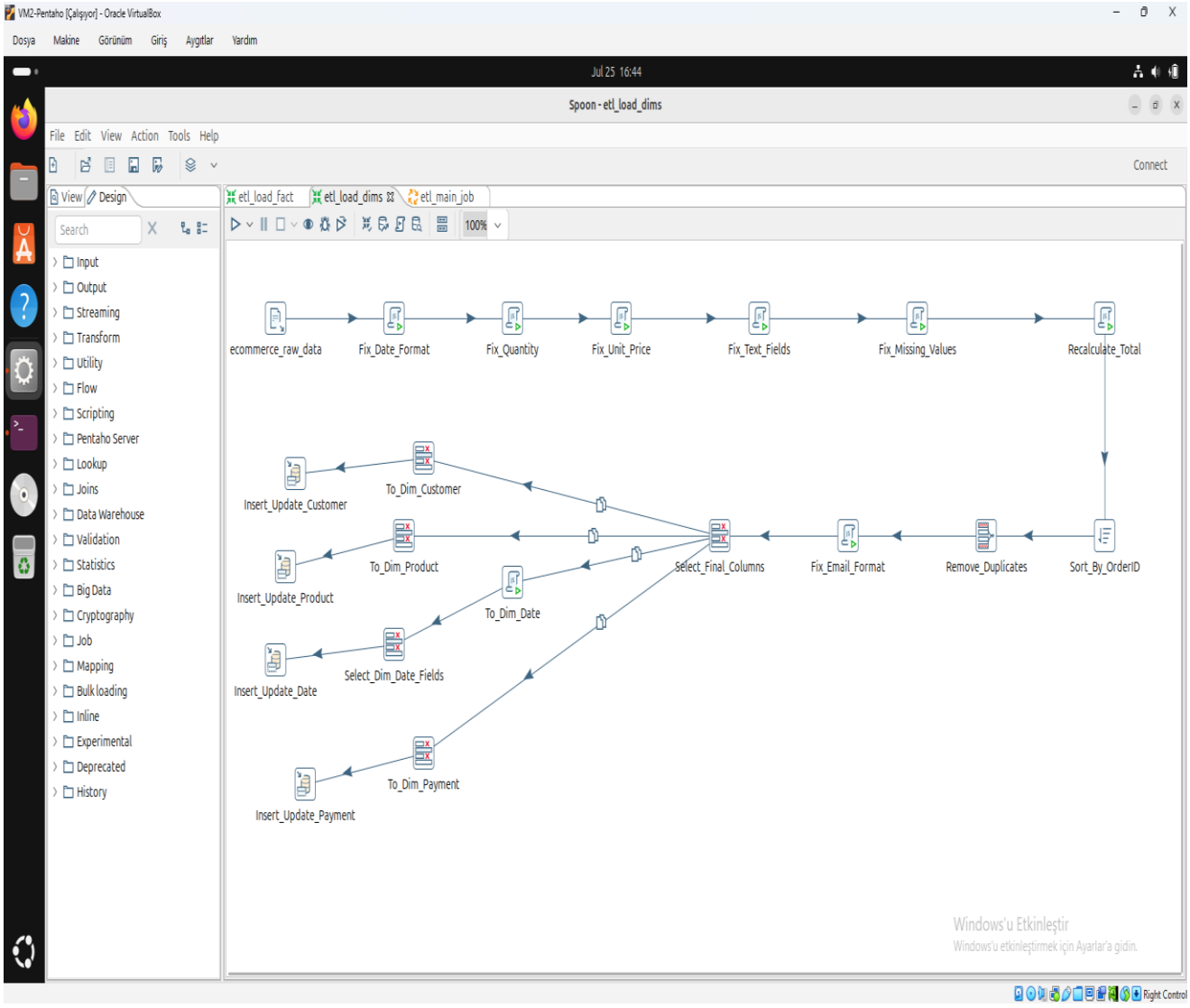
CREATE TABLE fact_orders (
    order_id VARCHAR(50) PRIMARY KEY,
    customer_id INT REFERENCES dim_customer(customer_id),
    product_id INT REFERENCES dim_product(product_id),
    date_id INT REFERENCES dim_date(date_id),
    payment_id INT REFERENCES dim_payment(payment_id),
    quantity INT,
    unit_price NUMERIC(12,2),
    total_amount NUMERIC(12,2),
    shipping_city VARCHAR(100),
    order_status VARCHAR(50)
);
```

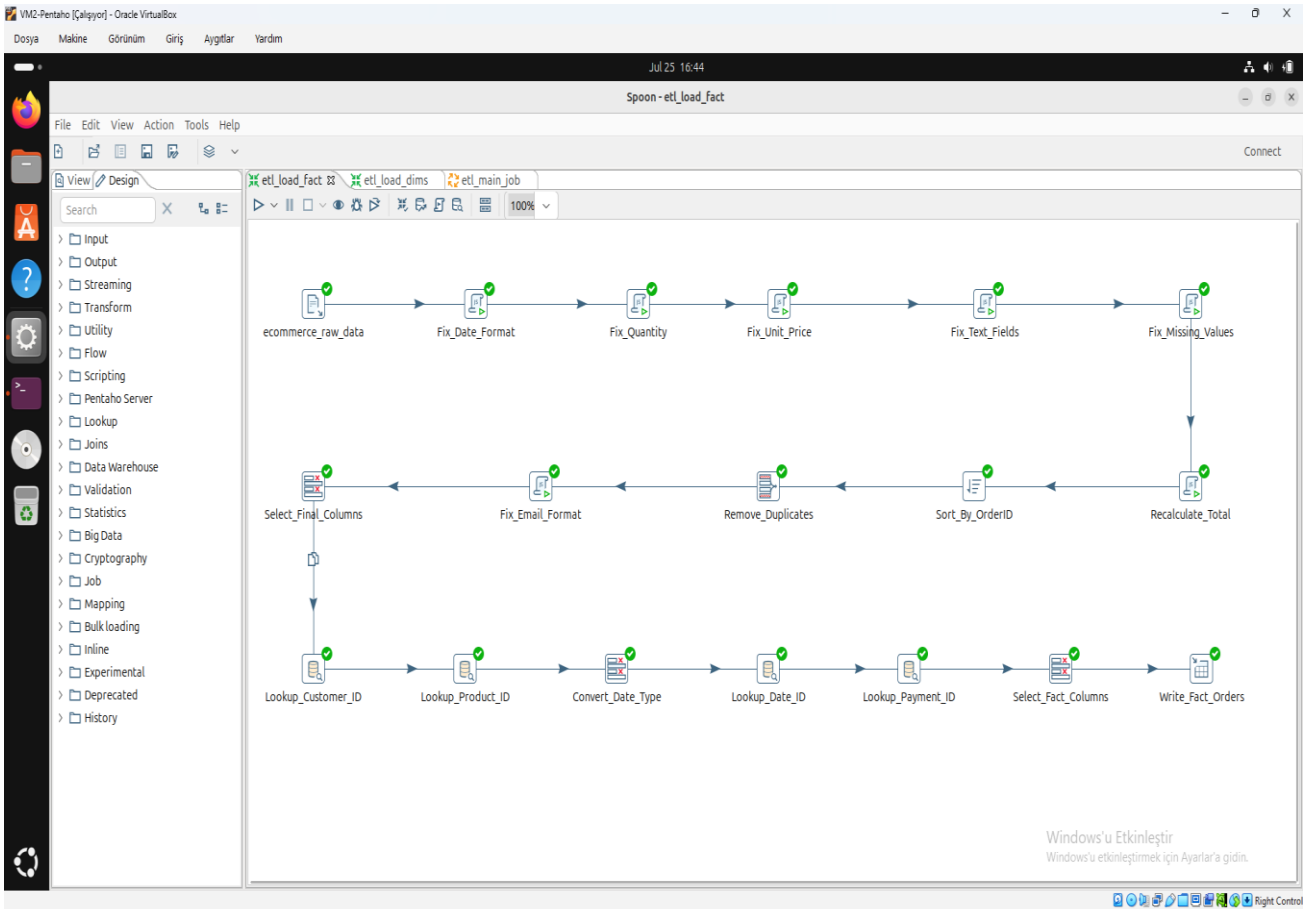
TRANSFORMATION:

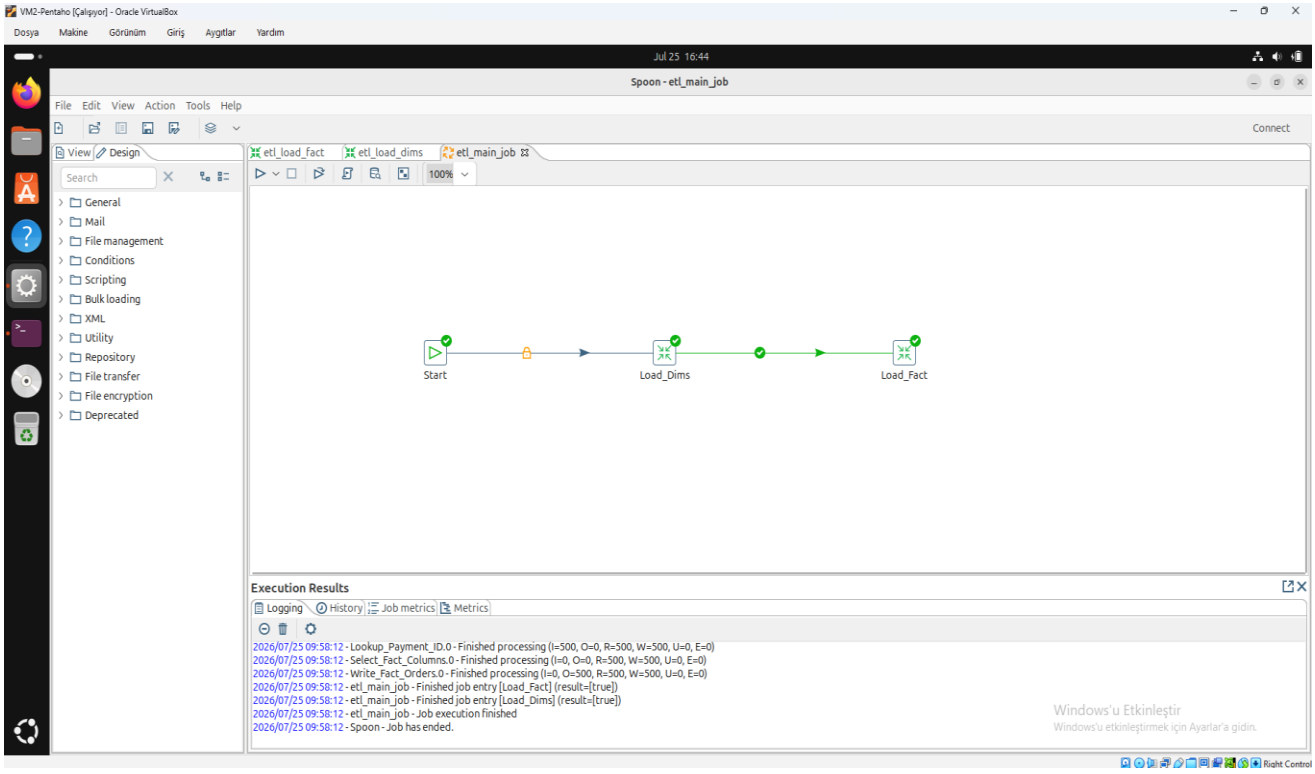
Gerçek hayatta Transform, ETL sürecinin en çok emek ve zaman gerektiren aşamasıdır. Genel bir kural olarak, bir ETL projesinde harcanan zamanın büyük kısmı (bazı kaynaklara göre yüzde 60'ın üzerinde) bu aşamada geçer. Bunun nedeni, Extract aşamasında sadece veriyi taşımanın yeterli olmasına karşın, Transform aşamasında verinin gerçekten anlaşılması, içindeki tutarsızlıkların tek tek fark edilip düzeltilmesi gerekmesidir.

Budüzeltme işlemleri gerçek dünyada genelde tek seferde değil, birbirini izleyen küçük adımlar halinde kurulur. Her adımın tek bir sorumluluğu vardır (örneğin sadece tarih formatını düzeltmek ya da sadece boş alanları doldurmak); bu sayede bir sorun çıktığında hangi adımın hataya neden olduğu kolayca tespit edilebilir.

Bizim projemizde de Transform aşaması, tam olarak bu mantıkla kuruldu. Pentaho Spoon üzerinde, her biri tek bir sorumluluğu olan 29 adımdan oluşan bir transformation akışı hazırlandı. Bu akış, CSV okuma ile başlayıp, alan bazlı temizlik adımlarından geçip, SQL ile oluşturduğumuz dört boyut tablosuna ve bir olgu tablosuna veri dağıtımıyla sona erdi.

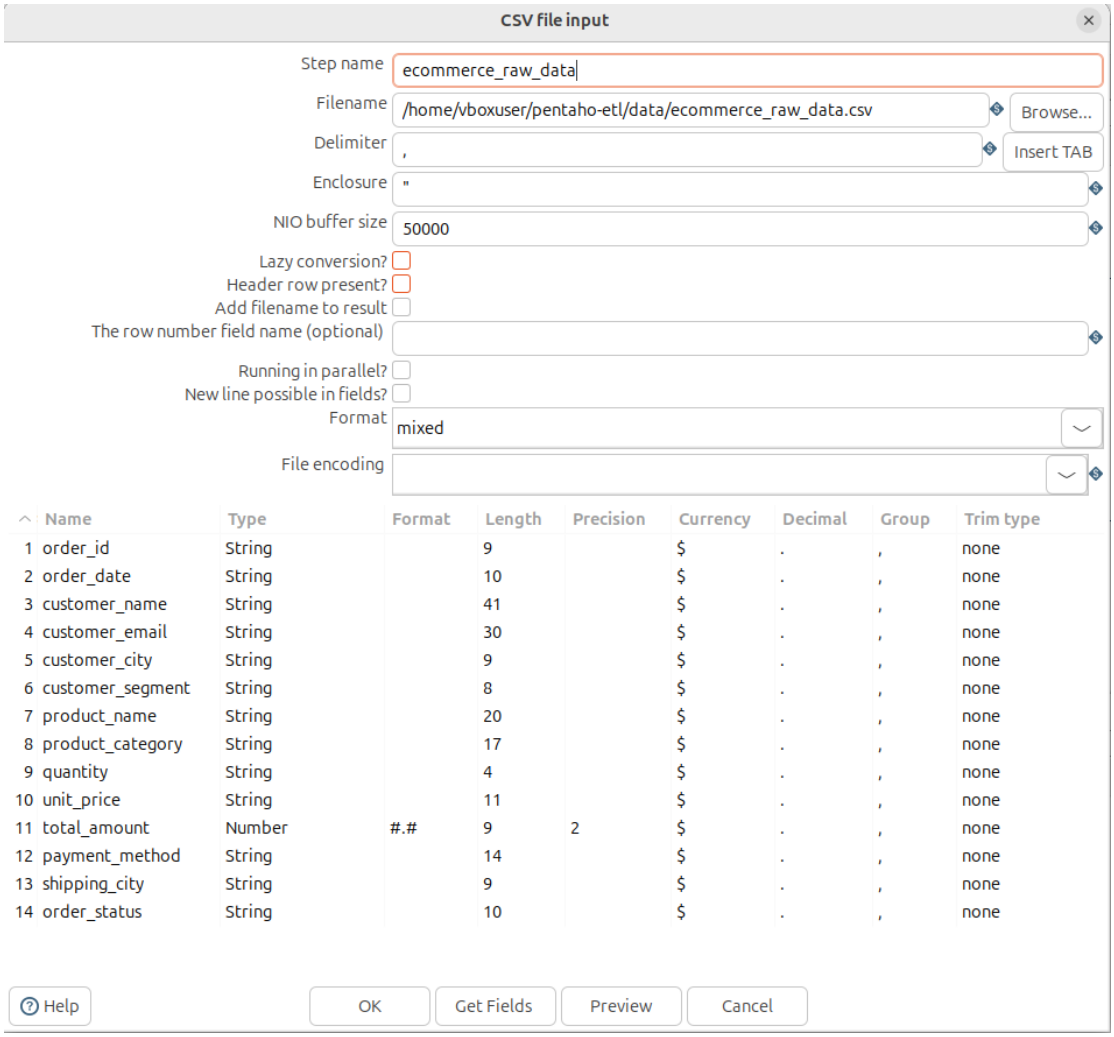



VM2-Pentaho [Çalışıyor] - Oracle VirtualBox
Dosya Makine Görünüm Giriş Aygıtlar Yardım
Jul 25 16:44
Spoon - etl_load_fact
File Edit View Action Tools Help
Connect
View Design
Search
Input
Output
Streaming
Transform
Utility
Flow
Scripting
Pentaho Server
Lookup
Joins
Data Warehouse
Validation
Statistics
Big Data
Cryptography
Job
Mapping
Bulk loading
Inline
Experimental
Deprecated
History
etl_load_fact
etl_load_dims
etl_main_job
100%
ecommerce_raw_data
Fix_Date_Format
Fix_Quantity
Fix_Unit_Price
Fix_Text_Fields
Fix_Missing_Values
Recalculate_Total
Sort_By_OrderID
Remove_Duplicates
Fix_Email_Format
Select_Final_Columns
Lookup_Customer_ID
Lookup_Product_ID
Convert_Date_Type
Lookup_Date_ID
Lookup_Payment_ID
Select_Fact_Columns
Write_Fact_Orders
Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.
Right Control


VM2-Pentaho [Çalışıyor] - Oracle VirtualBox
Dosya Makine Görünüm Giriş Aygıtlar Yardım
Jul 25 16:44
Spoon - etl_main_job
File Edit View Action Tools Help
Connect
View Design
Search
General
Mail
File management
Conditions
Scripting
Bulk loading
XML
Utility
Repository
File transfer
File encryption
Deprecated
etl_load_fact
etl_load_dims
etl_main_job
100%
Start
Load_Dims
Load_Fact
Execution Results
Logging
History
Job metrics
Metrics
2026/07/25 09:58:12 - Lookup_Payment_ID.0 - Finished processing (I=500, O=0, R=500, W=500, U=0, E=0)
2026/07/25 09:58:12 - Select_Fact_Columns.0 - Finished processing (I=0, O=0, R=500, W=500, U=0, E=0)
2026/07/25 09:58:12 - Write_Fact_Orders.0 - Finished processing (I=0, O=500, R=500, W=500, U=0, E=0)
2026/07/25 09:58:12 - etl_main_job - Finished job entry [Load_Fact] (result=[true])
2026/07/25 09:58:12 - etl_main_job - Finished job entry [Load_Dims] (result=[true])
2026/07/25 09:58:12 - etl_main_job - Job execution finished
2026/07/25 09:58:12 - Spoon - Job has ended.
Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.
Right Control

Şimdi etl_load_dims dosyamızla başlayalım.

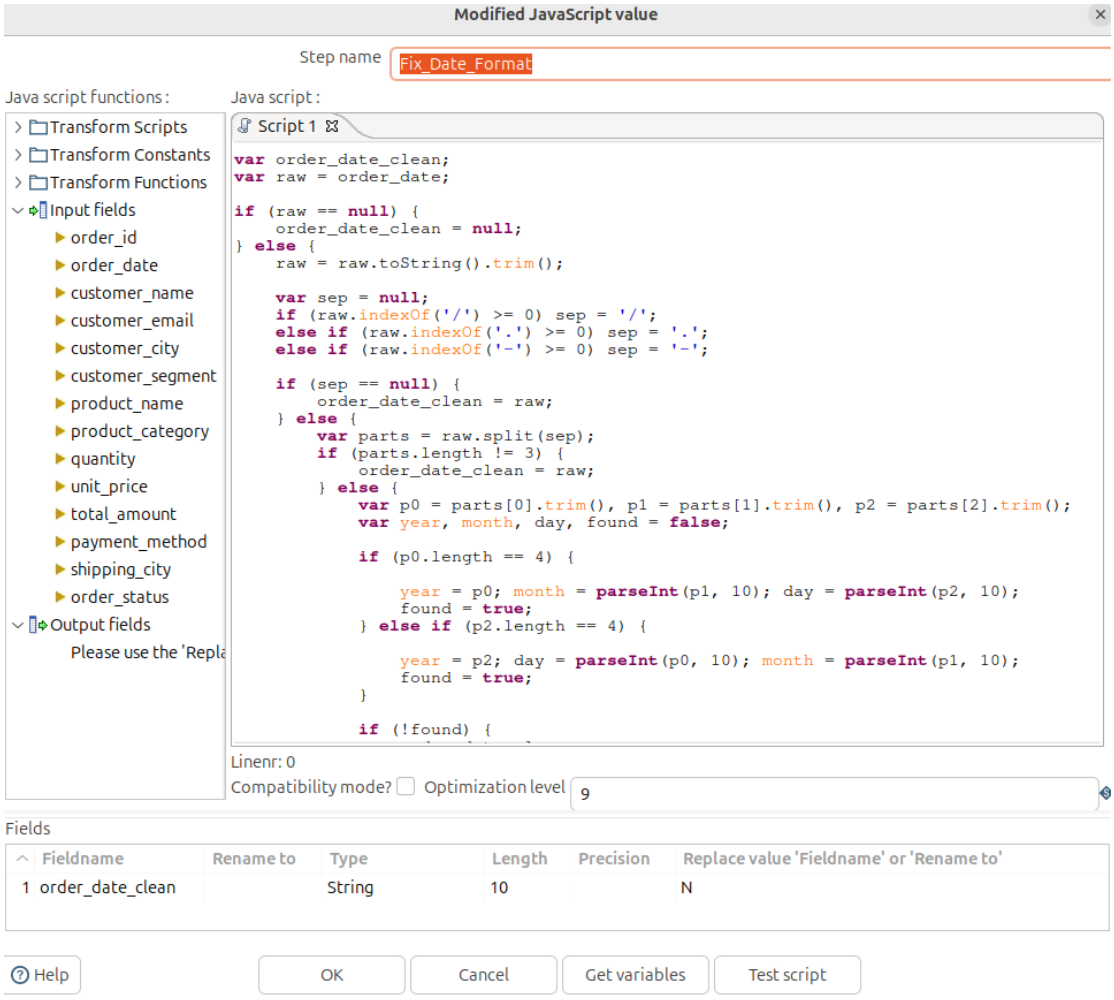
1. adım:



Ham CSV dosyası (ecommerce_raw_data.csv) transformation'a giriş noktası olarak tanımlandı. Delimiter virgül, Enclosure çift tırnak olarak ayarlandı; bu sayede "Roman Seti , 5 Kitap" gibi virgül içeren ürün isimleri bozulmadan okunabildi. Header row present? ve Lazy conversion? kutucukları işaretlendi. Bu sayede sütunlar Get Fields ile dosyadan otomatik okundu.

Alan tiplerinde quantity ve unit_price gibi sayısal olması beklenen sütunlar bilinçli olarak String bırakıldı, çünkü ham veride bu alanlarda yazıyla yazılmış ("iki" gibi) ya da negatif değerler bulunuyordu; doğrudan Number seçilseydi transformation daha ilk adımda hata verirdi. Sadece total_amount alanı, formatı daha tutarlı olduğu için Number olarak tanımlandı.

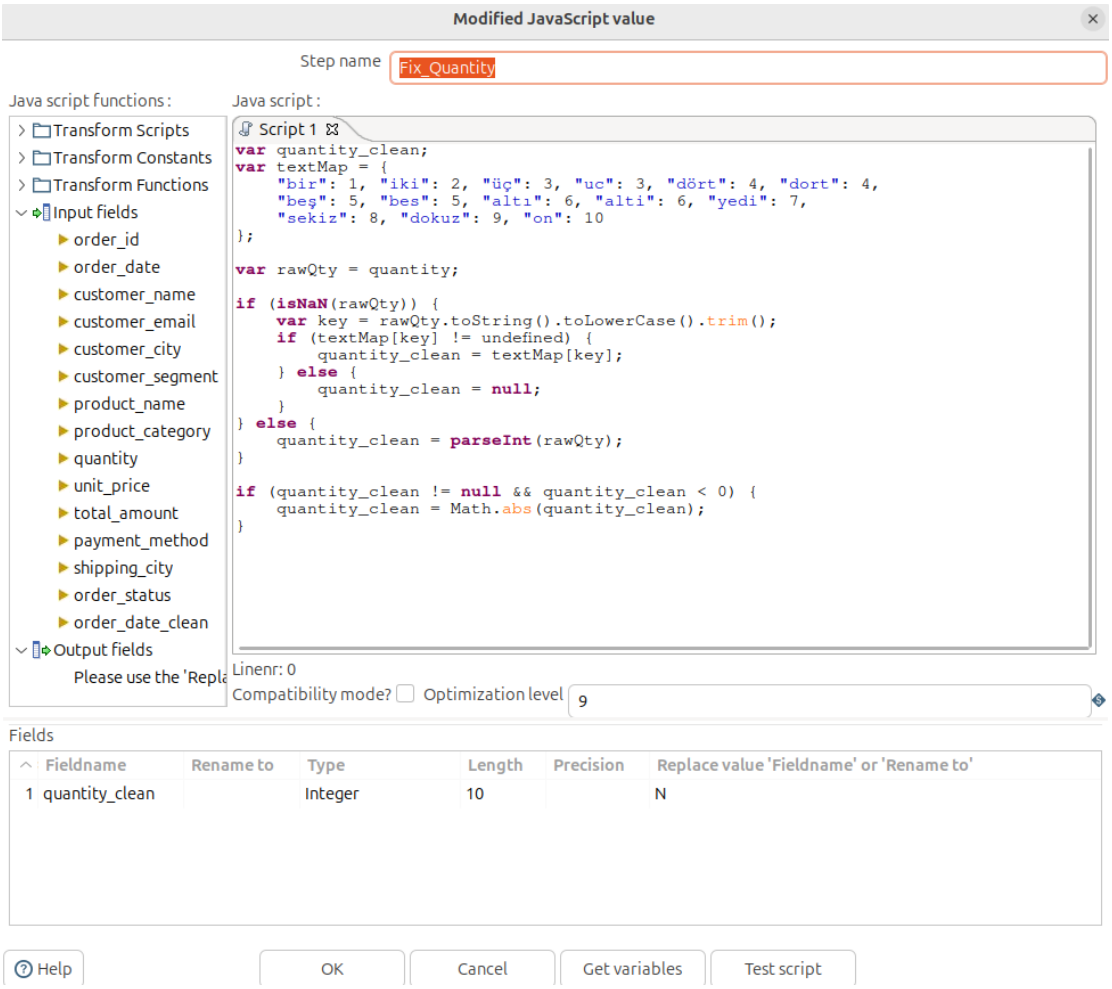
2. Adım:



Bu adım, order_date alanındaki farklı formatlardaki tarihleri (/, . veya - ayırıcılı, gün-ay-yıl ya da yıl-ay-gün sıralı) tek bir standarda (YYYY-AY-GUN) dönüştüren bir Modified JavaScript Value bileşenidir. Kod önce tarihi ayırıcıya göre üç parçaya böler, ardından yılın baştaki mi sondaki mi parçada olduğuna bakarak (parça uzunluğu 4 ise) gün, ay, yıl konumlarını belirler. Bulunamayan ya da tanınamayan formatlar (yıl hiçbir kenarda değilse) değiştirilmeden bırakılır.

Kodun en kritik kısmı, sonundaki güvenlik kontrolüdür: ay değeri 12'den büyük çıkıyor ve gün değeri 12'den küçükse, bu iki değerin ters yazıldığı anlaşılır ve otomatik olarak yer değiştirilir. Bu sayede gün-ay sırası karışık girilmiş tarihler de doğru şekilde düzeltilir. Sonuç, order_date_clean adında yeni bir String alanı olarak çıkışa eklenir; ham order_date alanına dokunulmaz.

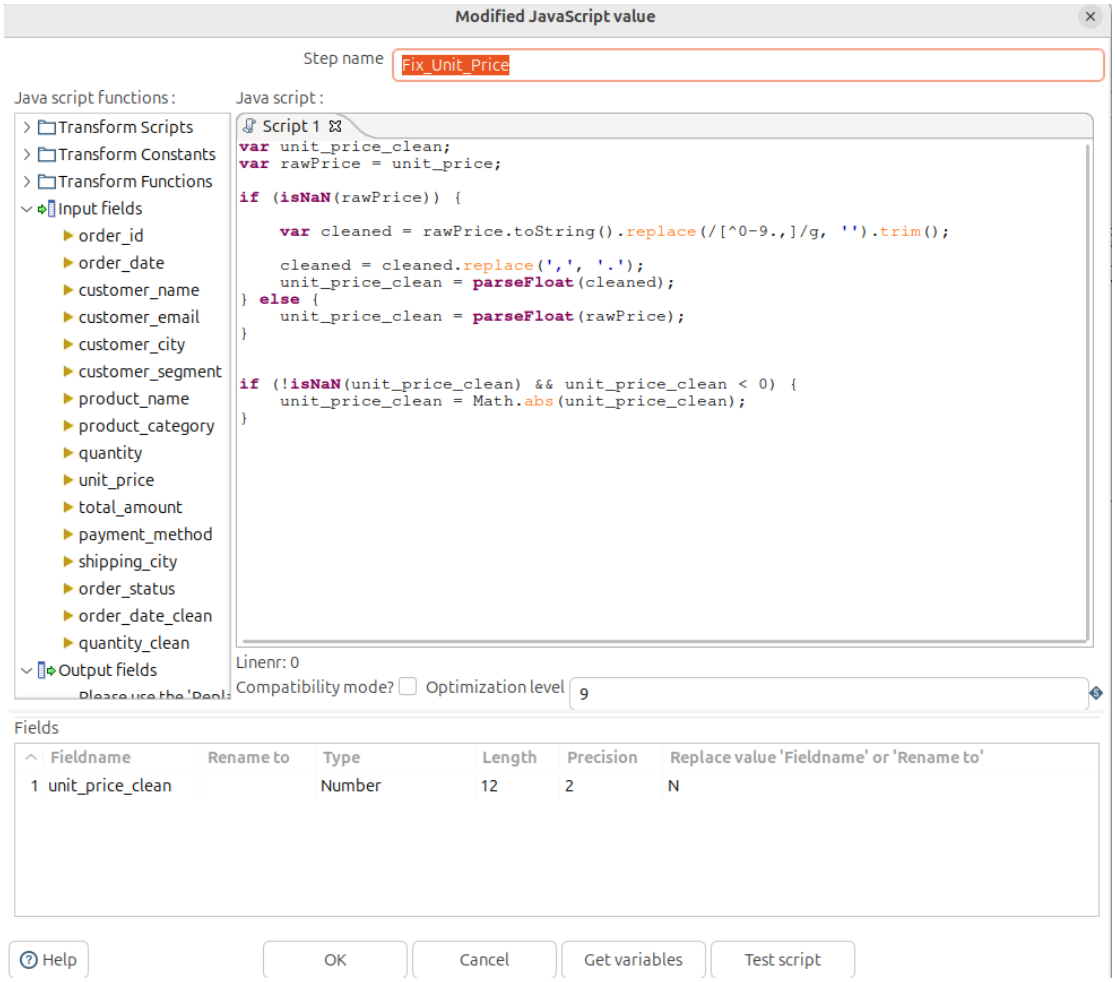
3. adım:



Bu adım, quantity alanındaki iki farklı sorunu (yazıyla girilmiş adetler ve negatif değerler) düzelten bir Modified JavaScript Value bileşenidir. Kod, önce bir textMap sözlüğü tanımlar; bu sözlük "bir", "iki", "üç" gibi Türkçe sayı isimlerini karşılık gelen rakamlara eşler. isNaN(rawQty) kontrolüyle değerin sayı olup olmadığı test edilir; sayı değilse (yani yazıyla girilmişse) bu sözlükten karşılığı aranır, sözlükte yoksa alan null bırakılır. Değer zaten sayıysa doğrudan parseInt ile tam sayıya çevrilir.

Son adımda, çıkan sonuç negatifse Math.abs() ile mutlak değeri alınarak pozitife çevrilir. Sonuç, quantity_clean adında yeni bir Integer alanı olarak çıkışa eklenir.

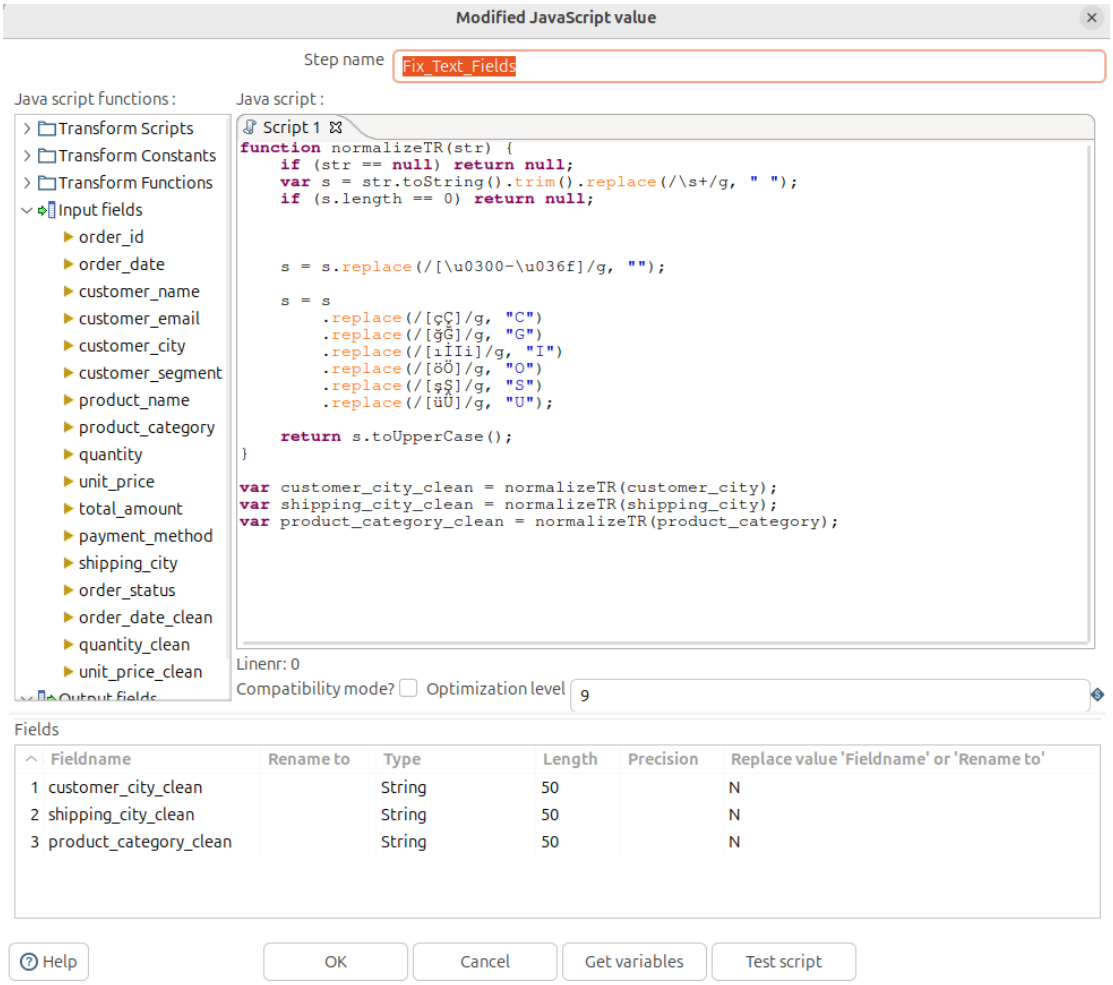
4. adım:



Bu adım, unit_price alanındaki biçim sorunlarını düzelten bir Modified JavaScript Value bileşenidir. Kod, önce isNaN(rawPrice) ile değerin doğrudan sayı olarak tanınıp tanınmadığını kontrol eder. Tanınmıyorsa, önce rakam ve virgül/nokta dışındaki tüm karakterler regex ile temizlenir, ardından virgüller noktaya çevrilerek (Türkçe ondalık gösteriminden standart ondalık gösterime) parseFloat ile sayıya dönüştürülür. Değer zaten düzgün bir sayıysa doğrudan parseFloat uygulanır.

Fix_Quantity adımında olduğu gibi burada da bir güvenlik kontrolü var: sonuç negatif çıkarsa Math.abs() ile pozitife çevrilir. Sonuç, unit_price_clean adında yeni bir Number alanı olarak çıkışa eklenir.

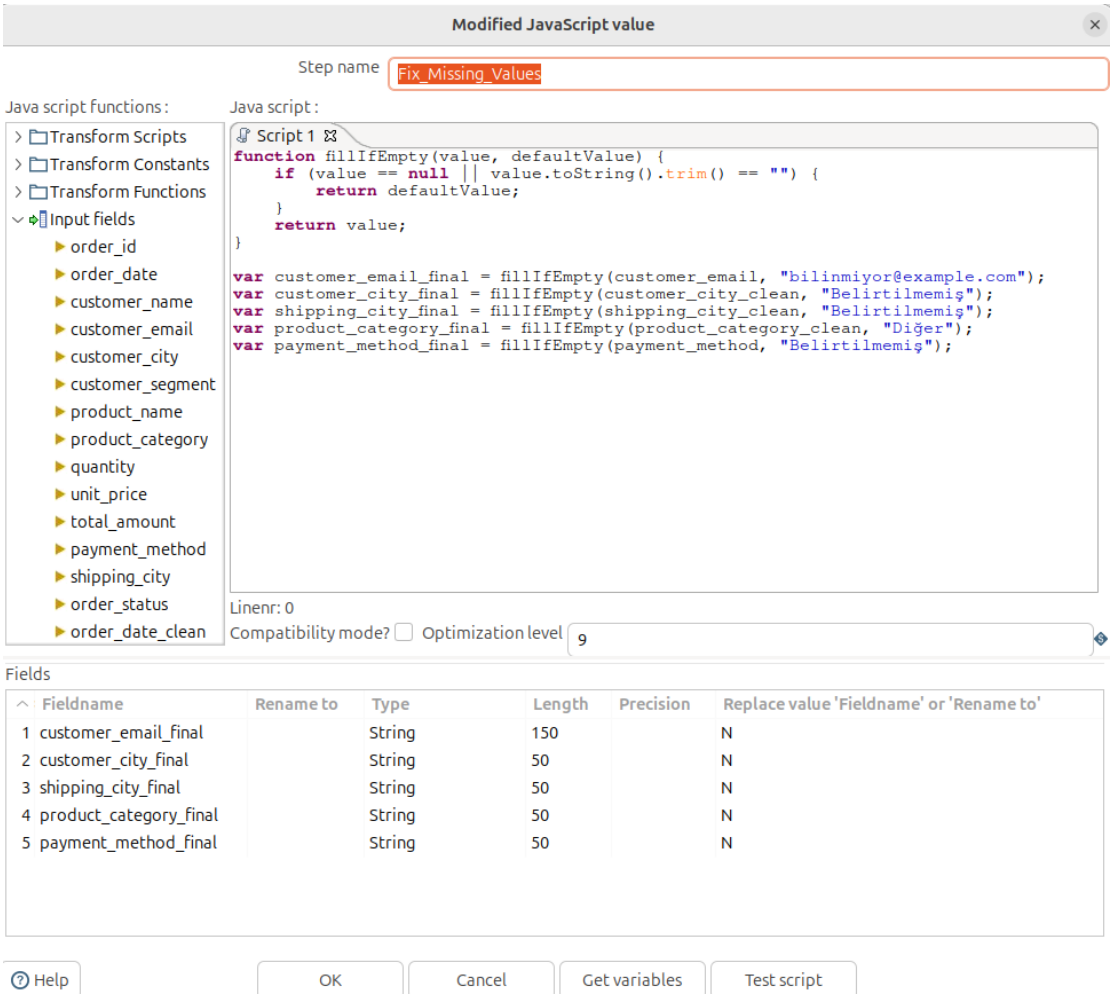
5. adım:



Bu adımda kullandığımız fonksiyon önce boşlukları (baş, son ve fazladan aradaki) temizler, ardından bir satırda karşılaşılan bozuk Unicode birleşik karakterleri (\u0300-\u036f aralığı, görünüşte tek karakter olan ama aslında iki karakterden oluşan "i" gibi yazımlar) kaldırır. Sonrasında Türkçe karakterleri (ç, ğ, ı/İ, ö, ş, ü) ASCII karşılıklarına çeviren bir dizi replace işlemi uygulanır ve sonuç toUpperCase() ile büyük harfe dönüştürülür.

Bu fonksiyon üç alana ayrı ayrı uygulanır: customer_city, shipping_city ve product_category. Sonuçlar sırasıyla customer_city_clean, shipping_city_clean ve product_category_clean adında üç yeni String alanı olarak çıkışa eklenir.

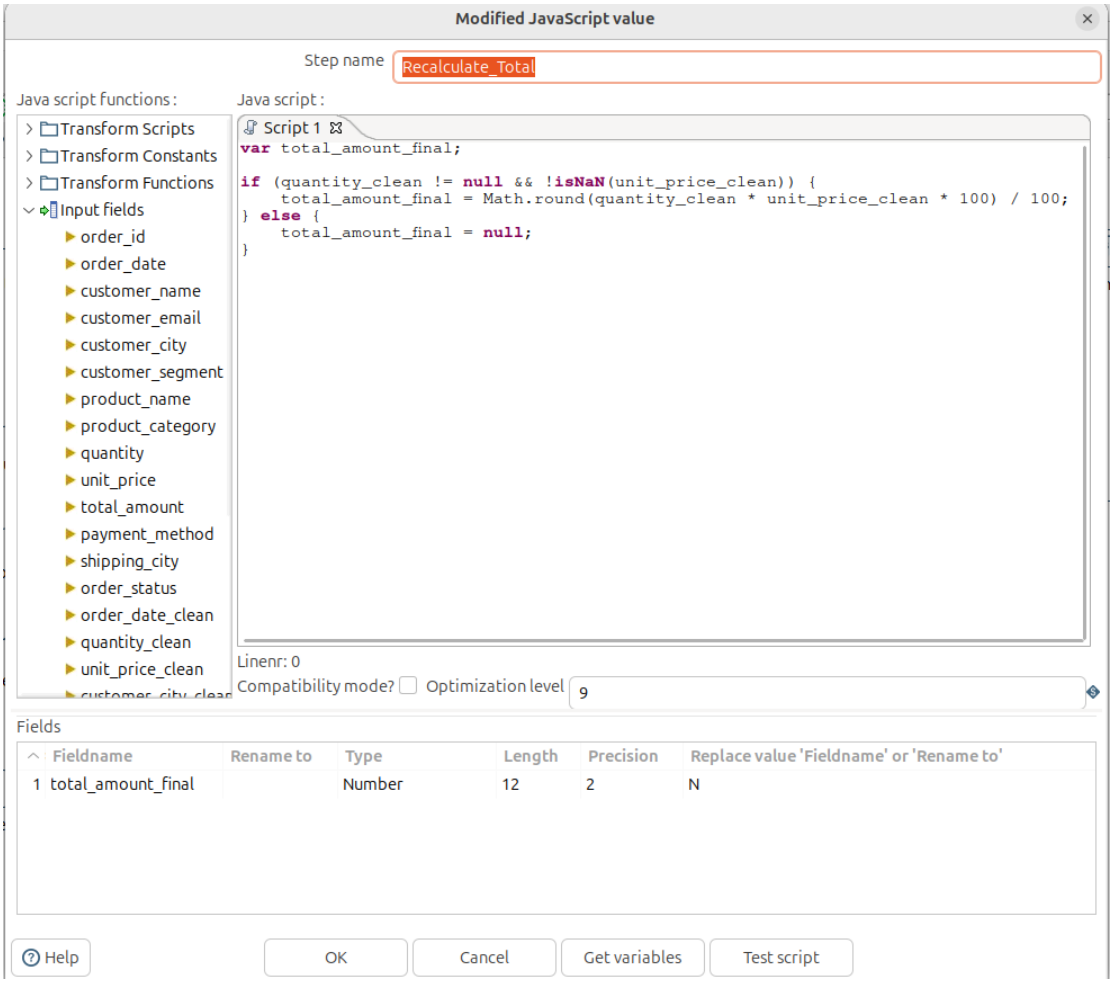
6. adım:



Bu adım, eksik bırakılmış alanları önceden belirlenmiş varsayılan değerlerle dolduran bir Modified JavaScript Value bileşenidir. Kod, fillIfEmpty adında genel bir yardımcı fonksiyon tanımlar; bu fonksiyon bir değerin null ya da boş metin olup olmadığını kontrol eder, boşsa kendisine verilen varsayılan değeri döndürür.

Bu fonksiyon beş alana uygulanır: customer_email boşsa "bilinmiyor@example.com", customer_city_clean ve shipping_city_clean boşsa "Belirtilmemiş", product_category_clean boşsa "Diğer", payment_method boşsa "Belirtilmemiş" değeriyle doldurulur. Sonuçlar customer_email_final, customer_city_final, shipping_city_final, product_category_final ve payment_method_final adında beş yeni alan olarak çıkışa eklenir. Bu adım, ileride yapılacak arama (lookup) işlemlerinde boş değerlerden kaynaklanan hataları önlemek için kritik bir güvenlik katmanıdır.

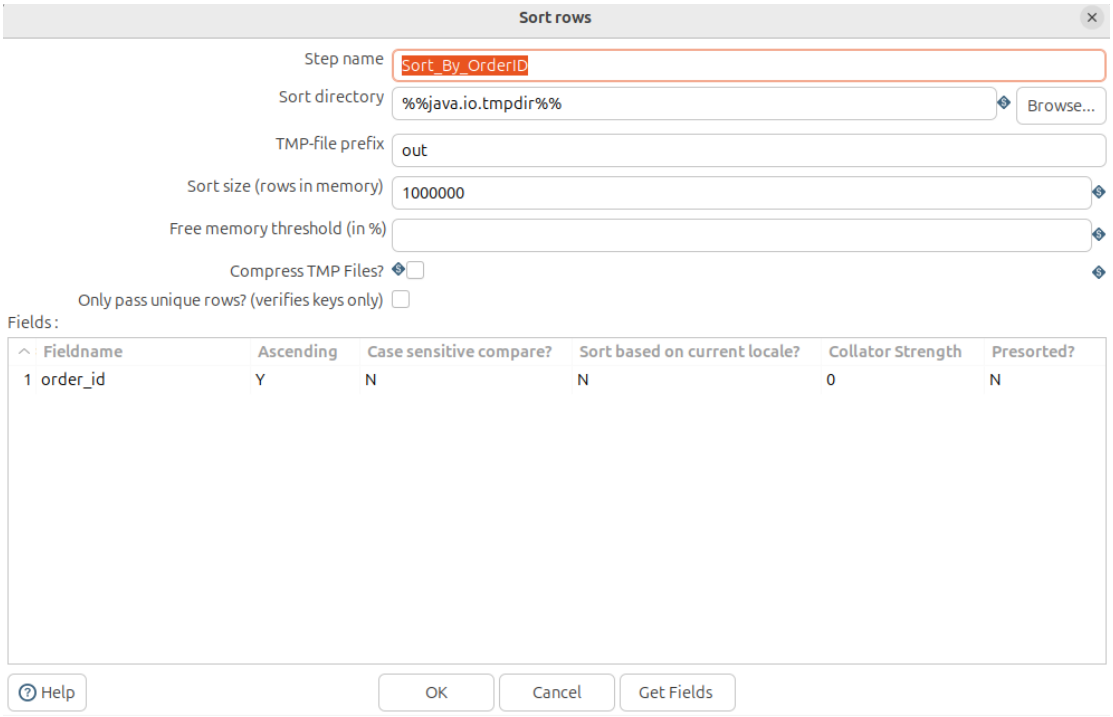
7. adım:



Bu adım, total_amount alanını daha önceki adımlarda düzeltilmiş quantity_clean ve unit_price_clean değerlerinden yeniden hesaplayan bir Modified JavaScript Value bileşenidir. Kod, önce bu iki alanın da geçerli (null olmayan ve sayısal) olup olmadığını kontrol eder; ikisi de geçerliyse çarpımları alınıp Math.round() ile iki ondalık basamağa yuvarlanır. Herhangi biri geçersizse total_amount_final null olarak bırakılır.

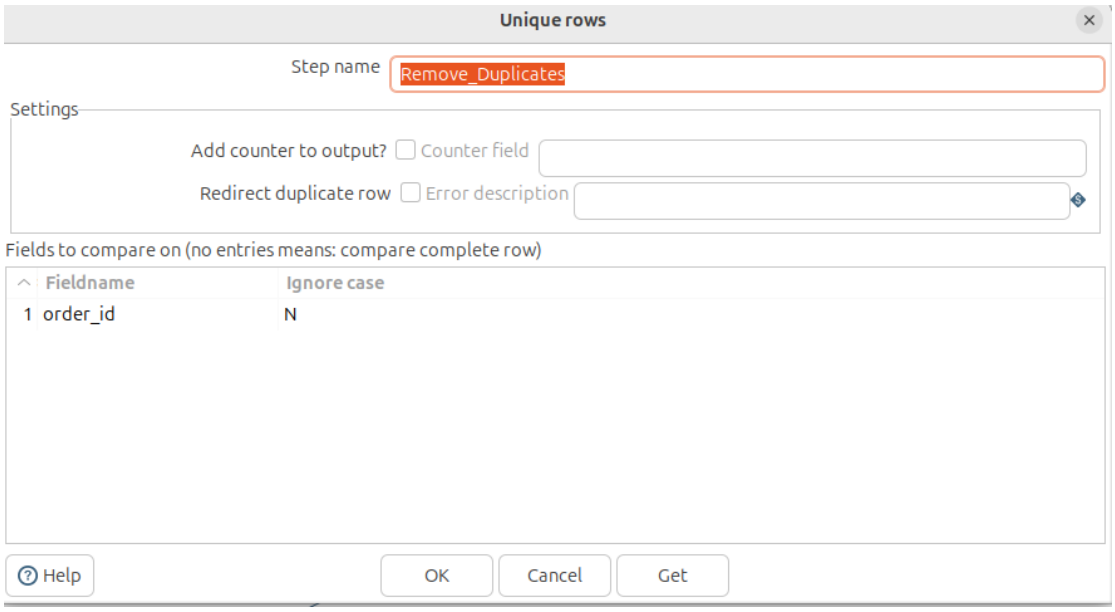
Bu adım, ham veride total_amount alanının hatalı ya da tutarsız girilmiş olabileceği ihtimaline karşı bir güvenlik katmanıdır; quantity ve unit_price düzeltildikten sonra, toplam tutarın da bu düzeltmelerle tutarlı hale gelmesini garanti eder.

8. adım:



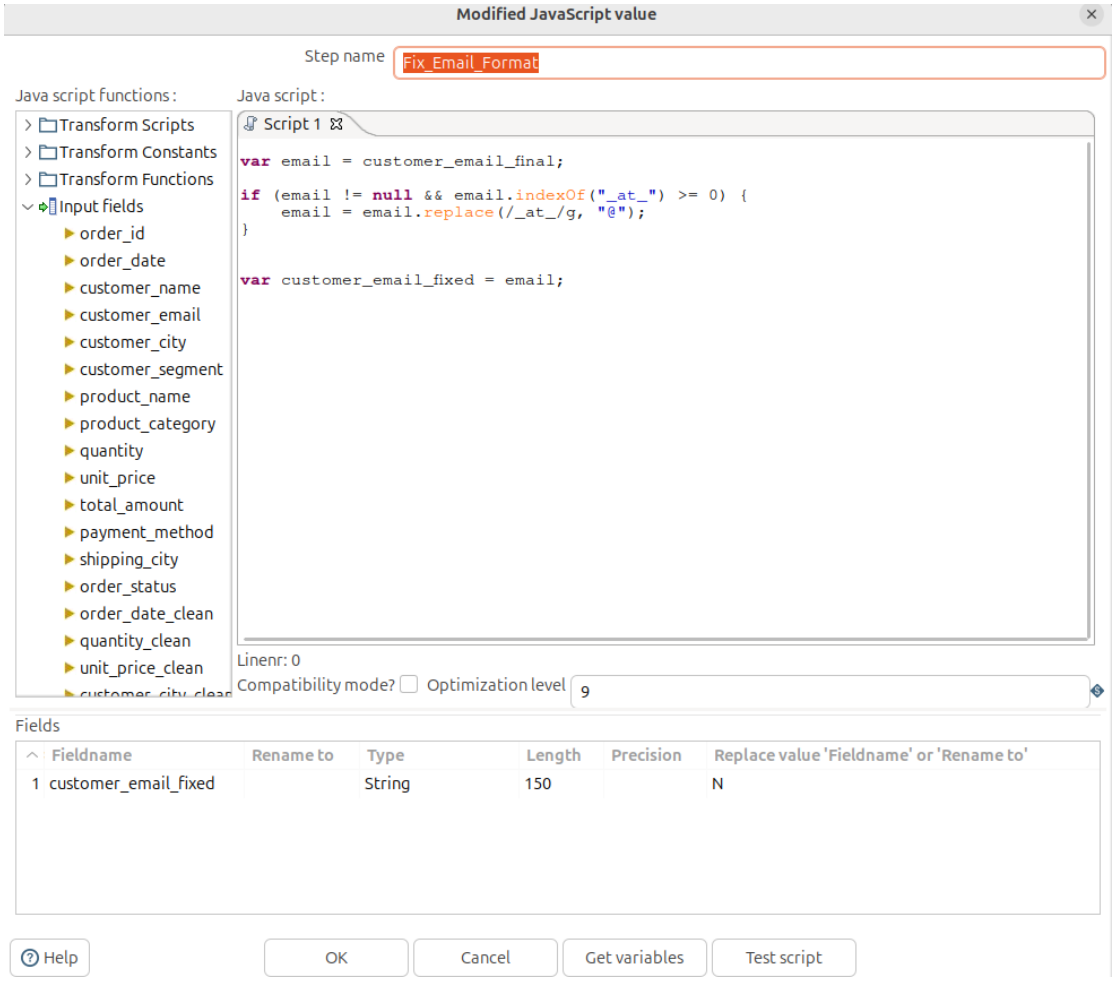
Bu adım, veriyi order_id alanına göre artan (Ascending: Y) sırada sıralayan bir Sort Rows bileşenidir. Sıralama, bir sonraki adımda yapılacak tekrar eden kayıt temizliğinin doğru çalışabilmesi için gereklidir; çünkü Pentaho'daki tekrar eden satır tespiti genelde ardışık (yan yana gelen) satırları karşılaştırarak çalışır, bu yüzden aynı order_id'ye sahip satırların yan yana gelmesi gerekir.

9. adım:



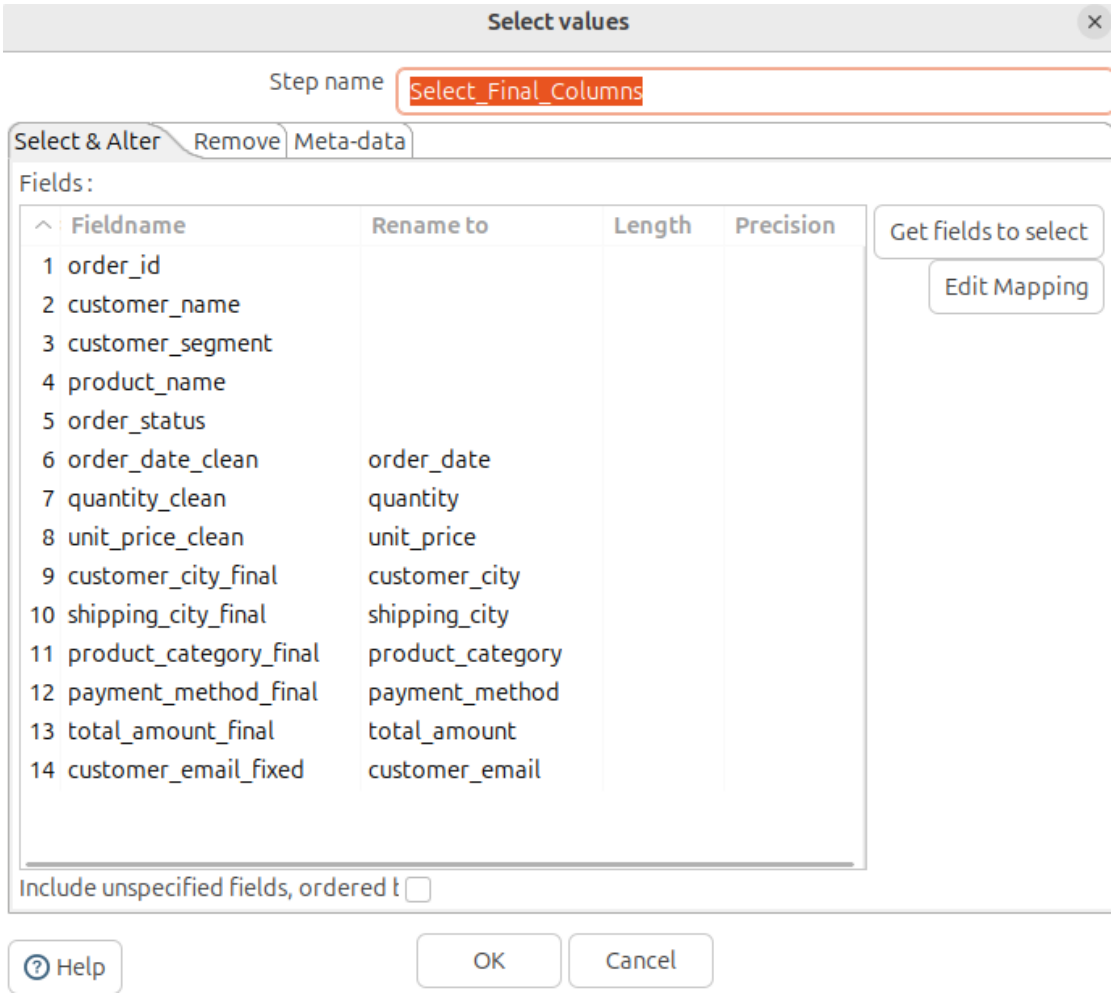
Bu adım, sıralanmış veri üzerinde order_id alanına göre tekrar eden satırları tespit edip kaldıran bir Unique Rows bileşenidir. Fields to compare on kısmında sadece order_id tanımlandığı için, karşılaştırma yalnızca bu alan üzerinden yapılır; aynı order_id'ye sahip ardışık satırlardan sadece ilki tutulur, sonrakiler elenir. Bu adım, ham veride aynı siparişin birden fazla kez kaydedilmiş olma ihtimaline karşı bir güvenlik adımıdır.

10. adım:



Bu adım, customer_email_final alanındaki bozuk e-posta formatını düzelten bir Modified JavaScript Value bileşenidir. Kod, e-posta içinde "at" ifadesi geçip geçmediğini kontrol eder (bir önceki bölümde bahsettiğimiz buyrukhanyuksel_at_example.net gibi hatalı yazımlar için); geçiyorsa bu ifadeyi standart "@" karakteriyle değiştirir. Sonuç, customer_email_fixed adında yeni bir String alanı olarak çıkışa eklenir.

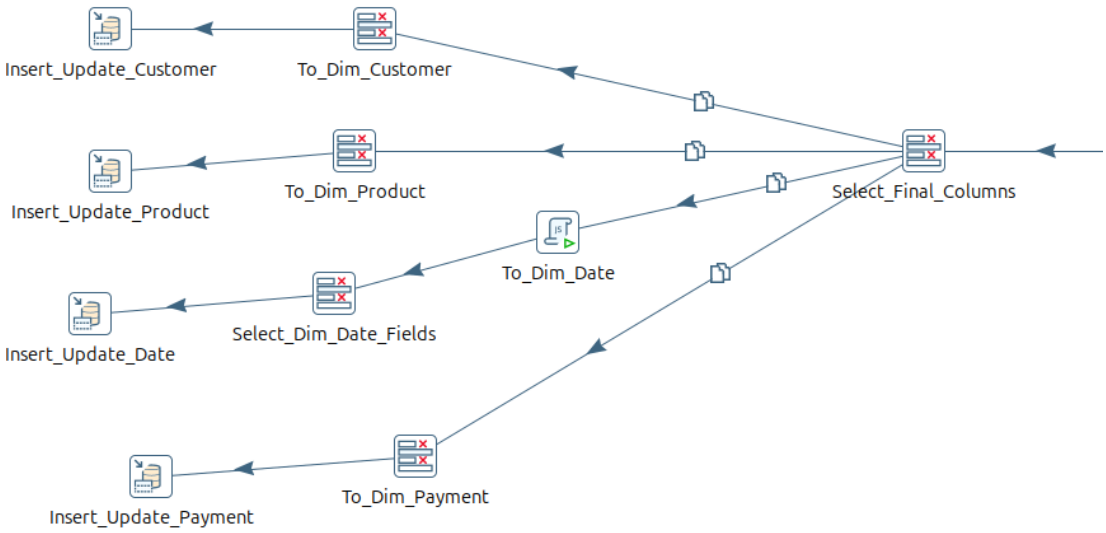
11. adım:



Bu adım, önceki adımlarda üretilen tüm "_clean" ve "_final" uzantılı geçici alanları toplayıp, orijinal alan isimleriyle yeniden adlandıran bir Select Values bileşenidir. Örneğin order_date_clean → order_date, quantity_clean → quantity, customer_city_final → customer_city şeklinde yeniden isimlendirilir. Bu sayede transformation'ın geri kalanında hem temiz hem sade bir alan seti kullanılır; ham ve ara adımlarda üretilen orijinal bozuk alanlar (order_date, quantity, customer_city gibi) burada listeye dahil edilmediği için akıştan düşürülmüş olur.

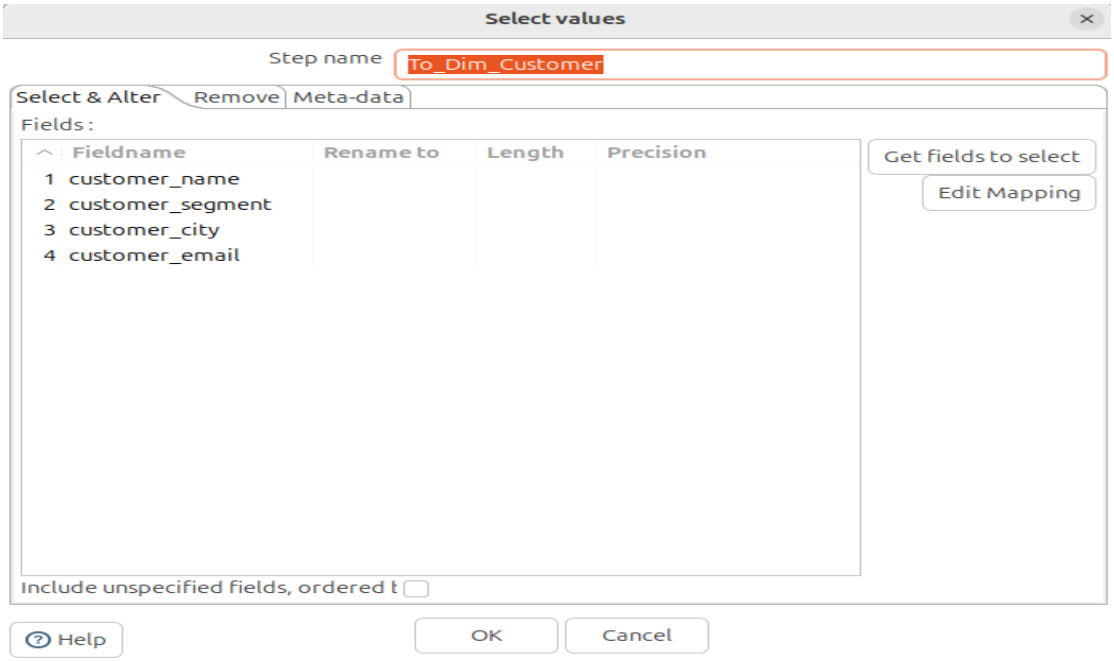
Bu adımdan sonra veri, artık tamamen temizlenmiş 14 alanlık son haliyle dört boyut tablosuna dağıtılmaya hazırdır.

12. adım:



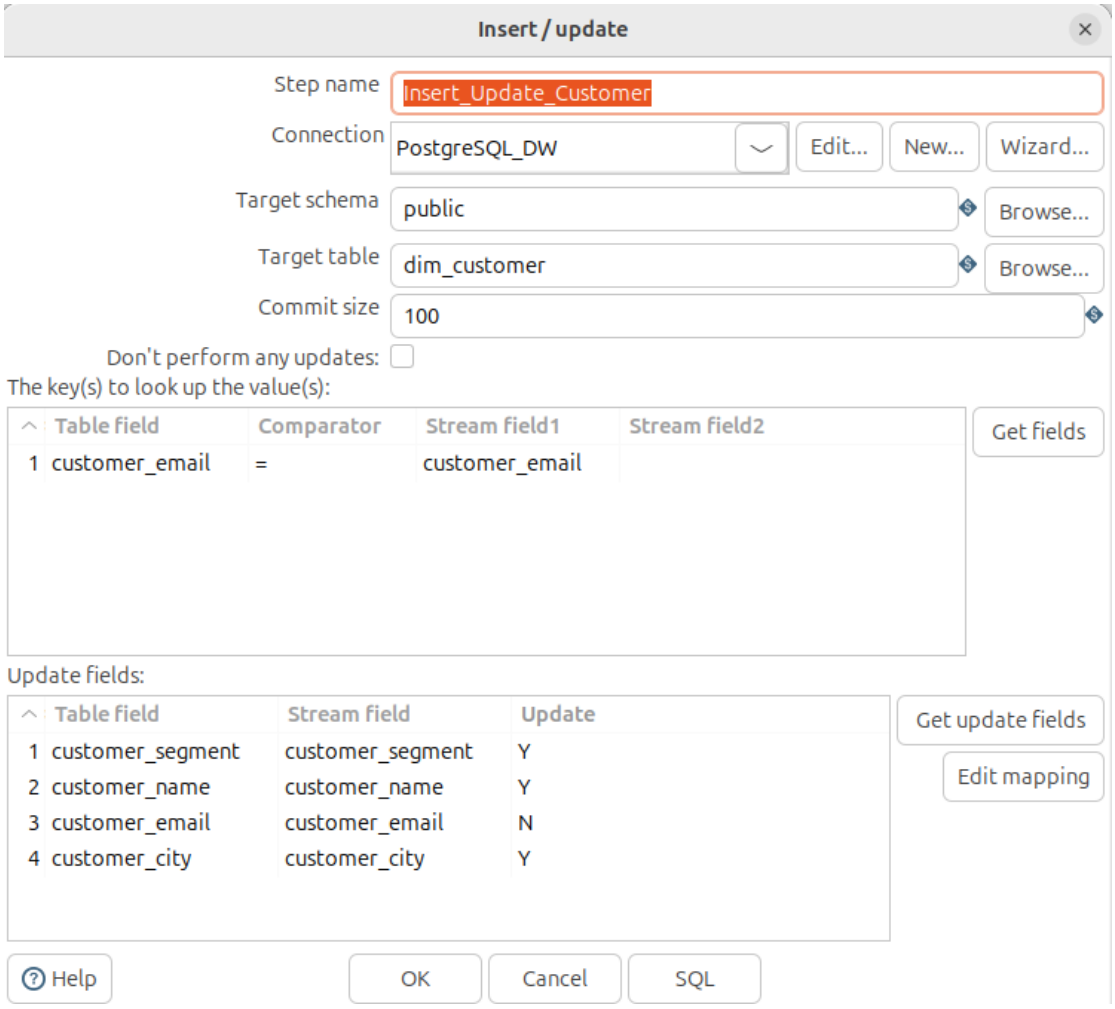
Select_Final_Columns adımından sonra, temizlenmiş ana veri seti eşzamanlı olarak kopyalanarak Yıldız Şeması'nın (Star Schema) boyut (dimension) tablolarını oluşturmak üzere dört farklı paralel kola dağıtılmaktadır. Her bir kolda öncelikle bir filtreleme adımı (To_Dim_...) kullanılarak sadece o hedef boyuta ait spesifik sütunlar (müşteri, ürün veya ödeme detayları) izole edilir ve gereksiz veriler atılır; tarih kolunda ise ekstra olarak bir JavaScript adımıyla (To_Dim_Date) gün, ay, yıl gibi zaman kırılımları üretilir. Son aşamada ise daraltılmış bu spesifik veri paketleri Insert / Update adımlarına iletilerek, veritabanındaki ilgili boyut tablolarına (dim_customer, dim_product, dim_date, dim_payment) tekrar eden kayıt yaratmayacak şekilde (upsert mantığıyla) kalıcı olarak yazılır.

12.1. adım:



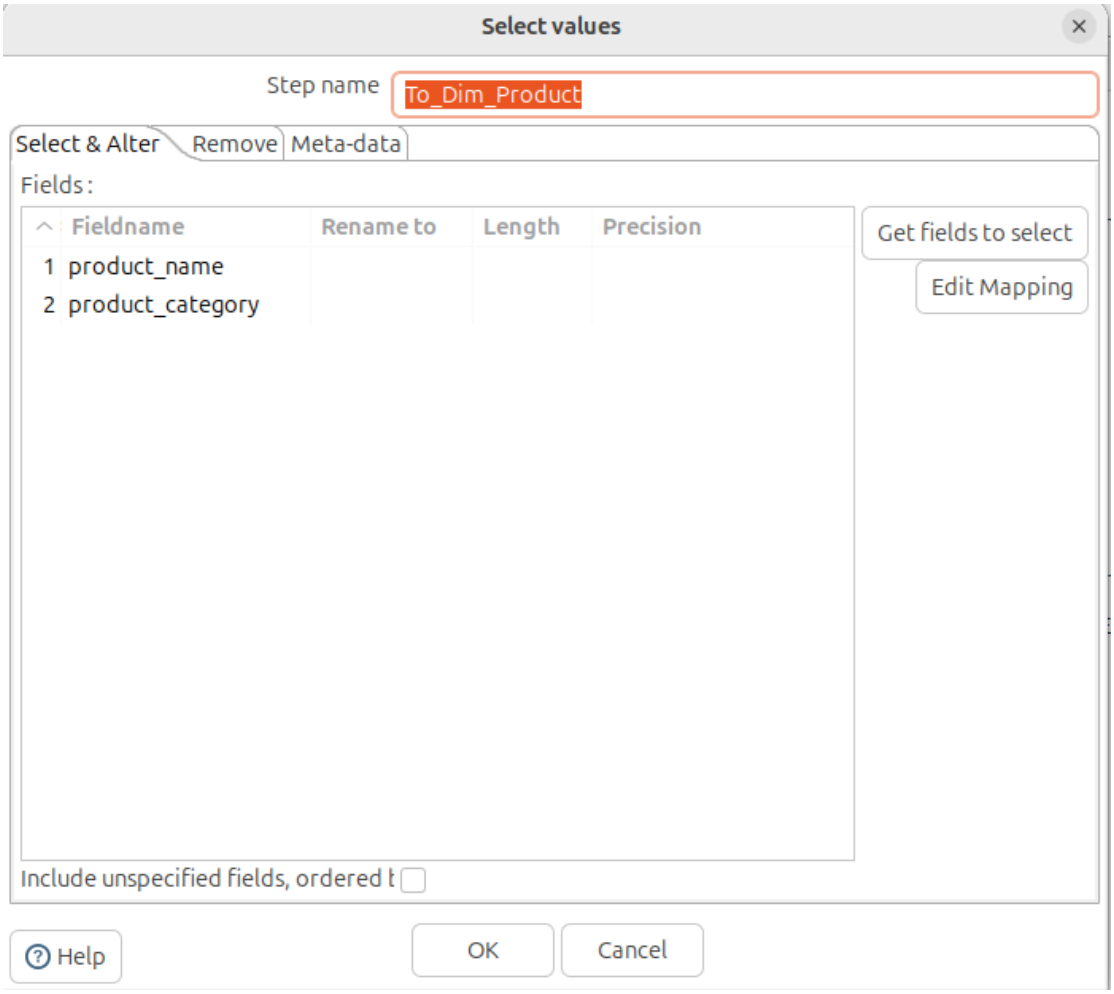
Bu adım, Select_Final_Columns'tan gelen temiz veri setinden yalnızca müşteri boyutuyla ilgili dört alanı (customer_name, customer_segment, customer_city, customer_email) seçen bir Select Values bileşenidir. Diğer tüm alanlar (ürün, tarih, ödeme bilgileri gibi) bu kolda elenir; çünkü bu kolun tek görevi dim_customer tablosuna yazılacak veriyi hazırlamaktır.

12.2. adım:



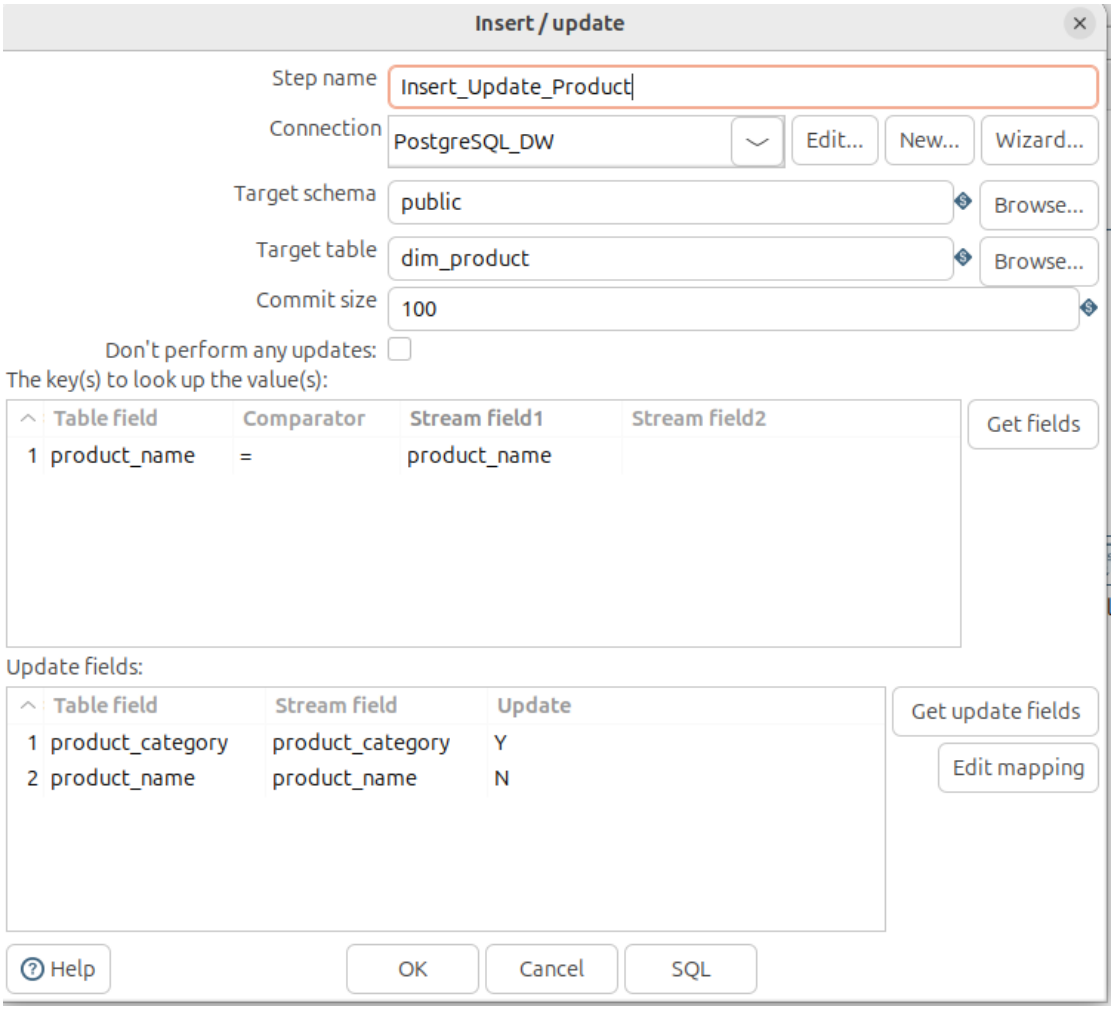
Bu adım, PostgreSQL_DW bağlantısı üzerinden dim_customer tablosuna veri yazan bir Insert/Update bileşenidir. Anahtar (key) olarak customer_email seçilmiştir: gelen her satırın e-posta adresi tabloda aranır, eşleşme bulunursa customer_segment, customer_name, customer_email ve customer_city alanları güncellenir (Update: Y); eşleşme bulunamazsa yeni bir müşteri satırı olarak eklenir. Bu upsert mantığı sayesinde aynı müşteriye ait farklı siparişler, dim_customer tablosunda tekrar satır oluşturmadan tek bir kayıt üzerinde birleşir.

12.3. adım:



Bu adım, temiz veri setinden yalnızca ürün boyutuyla ilgili iki alanı (product_name, product_category) seçen bir Select Values bileşenidir

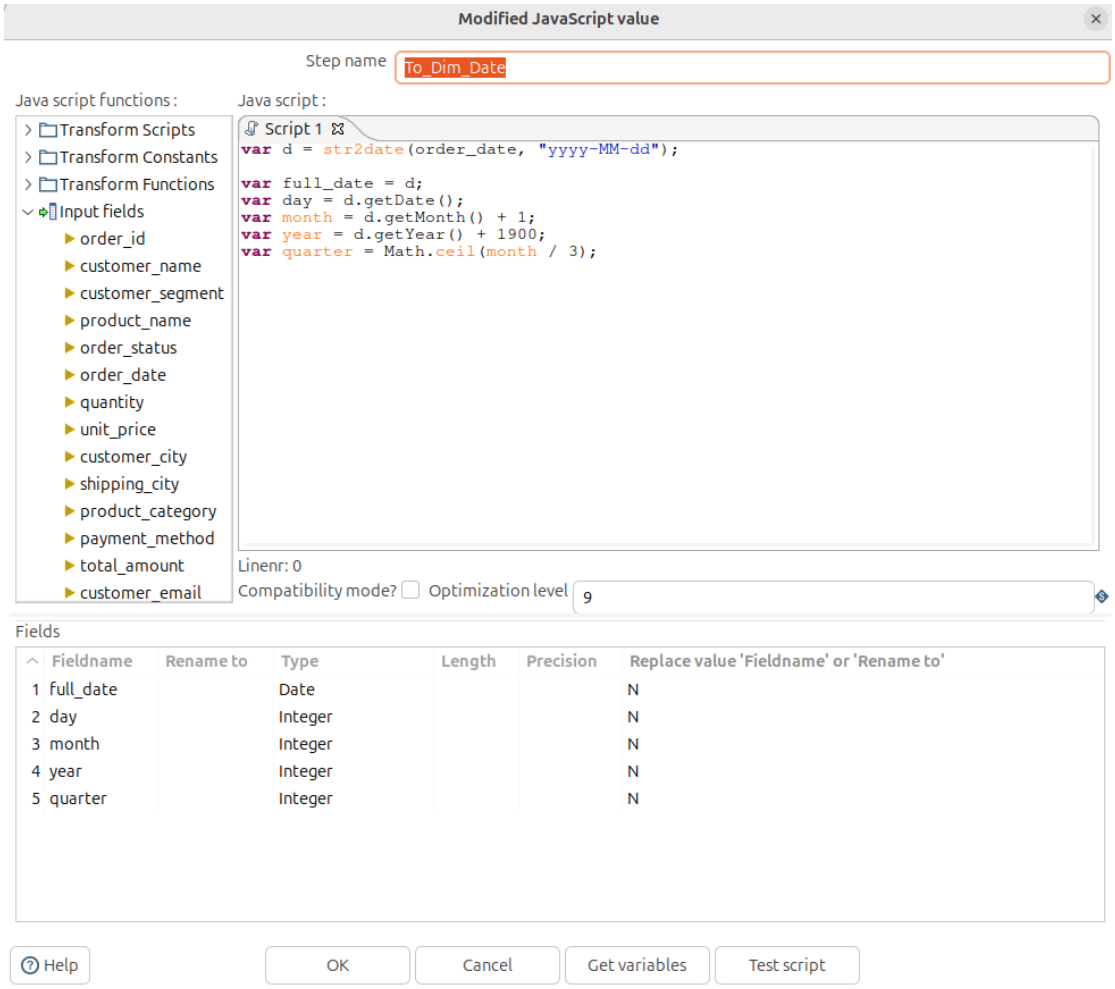
12.4. Adım:



Bu adım, dim_product tablosuna yazan Insert/Update bileşenidir ve daha önce bahsettiğimiz tekrar eden ürün kaydı sorununun asıl çözümünü barındırır. Anahtar (key) olarak yalnızca product_name tanımlanmıştır; product_category anahtara dahil edilmemiştir. Update fields kısmında ise product_category alanının Update değeri bilinçli olarak N (hayır) bırakılmıştır, yalnızca product_name Y (güncellenir) olarak işaretlenmiştir.

Bu ayarın anlamı şudur: bir ürün ilk kez görüldüğünde, o anki kategorisiyle birlikte tabloya eklenir. Aynı ürün daha sonra (kategorisi boş ya da farklı yazılmış olarak) tekrar geldiğinde, product_name üzerinden eşleşme bulunur ve satır güncellenmez, üzerine yeni bir kayıt da açılmaz; ürünün ilk görülen doğru kategorisi kalıcı olarak korunur. Bu değişiklik, dim_product tablosunu 34 satırdan gerçek ürün sayısı olan 20 satıra düşürmüştür.

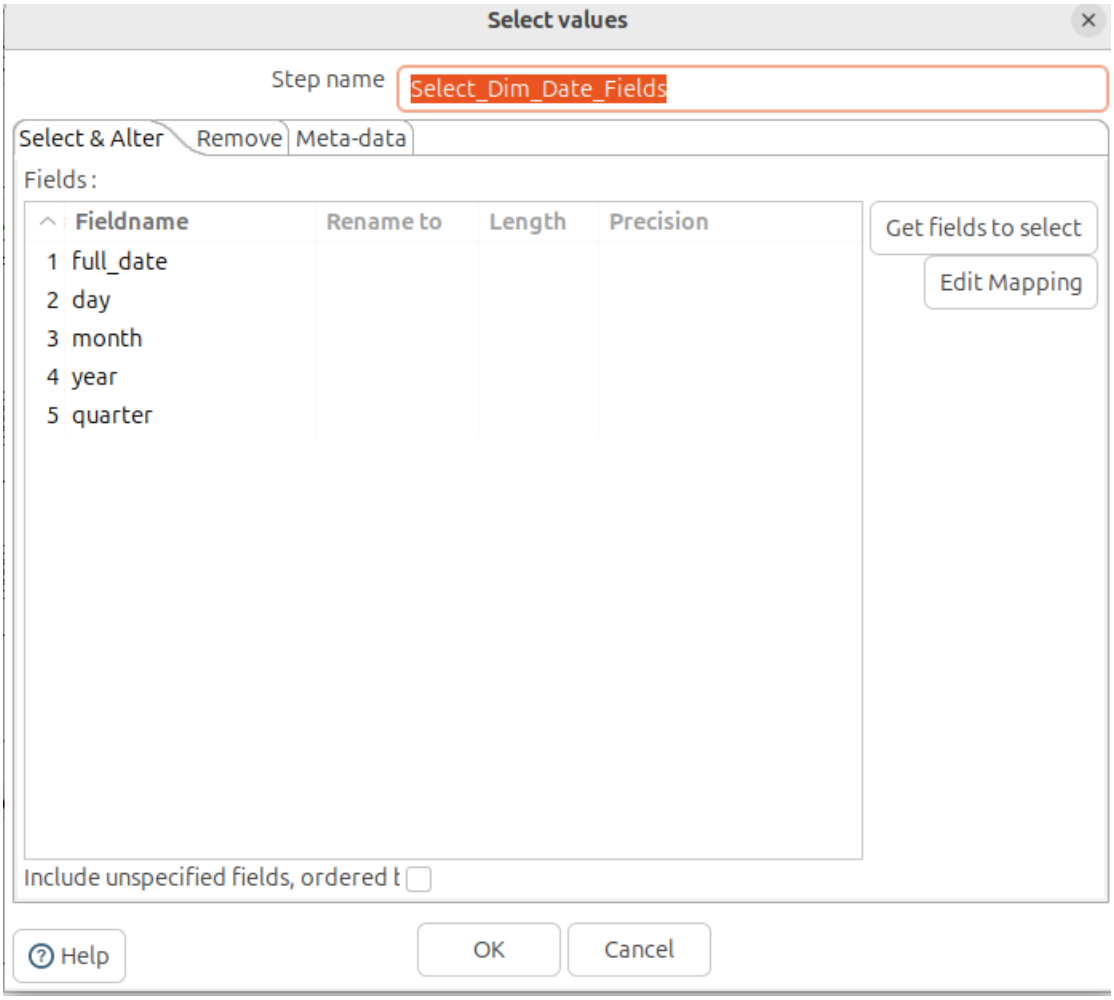
12.5. Adım:



Bu adım, order_date alanından tarih boyutuna ait kırılımları türeten bir Modified JavaScript Value bileşenidir. Kod önce str2date fonksiyonuyla order_date alanını (11. adımda "yyyy-MM-dd" formatına standartlaştırılmış haliyle) gerçek bir tarih nesnesine çevirir. Ardından bu tarih nesnesinden day, month, year ve quarter bilgileri türetilir.

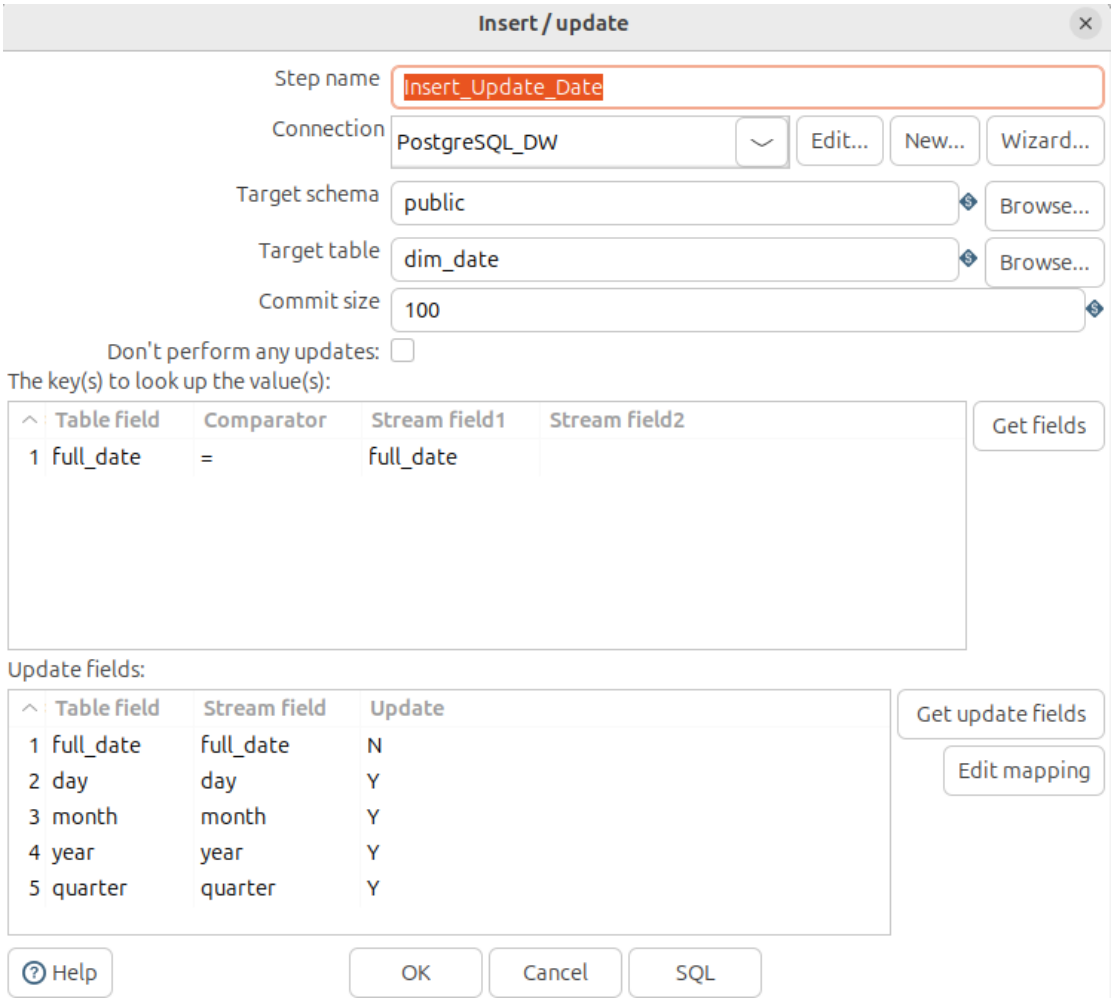
Burada dikkat edilmesi gereken bir nokta, getMonth() ve getYear() fonksiyonlarının JavaScript'in eski (legacy) tarih metodları olmasıdır: getMonth() ayı 0-11 aralığında döndürdüğü için sonuca +1 eklenmiş, getYear() ise yılı 1900'den itibaren saydığı için sonuca +1900 eklenerek gerçek yıl değeri (örneğin 2025) elde edilmiştir. quarter alanı ise ayın 3'e bölünüp yukarı yuvarlanmasıyla (Math.ceil) hesaplanmıştır. Sonuçta full_date, day, month, year ve quarter adında beş yeni alan çıkışa eklenir.

12.6. Adım:



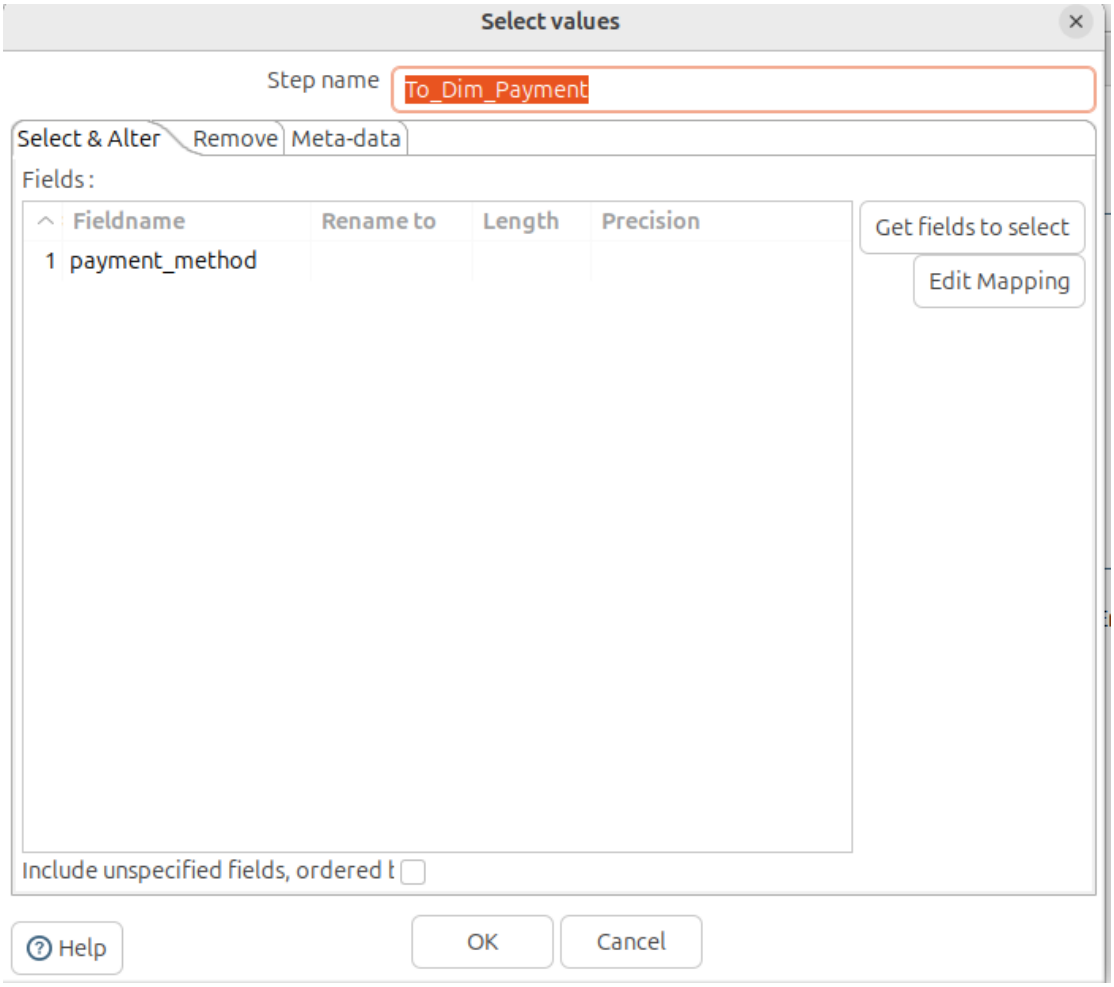
Bu adım, bir önceki adımda türetilen beş tarih alanını (full_date, day, month, year, quarter) seçip diğer tüm alanlardan (müşteri, ürün, sipariş bilgileri gibi) arındıran bir Select Values bileşenidir. Bu sayede dim_date tablosuna yalnızca tarih boyutuyla ilgili, sade bir alan seti iletilmiş olur.

12.7. Adım:



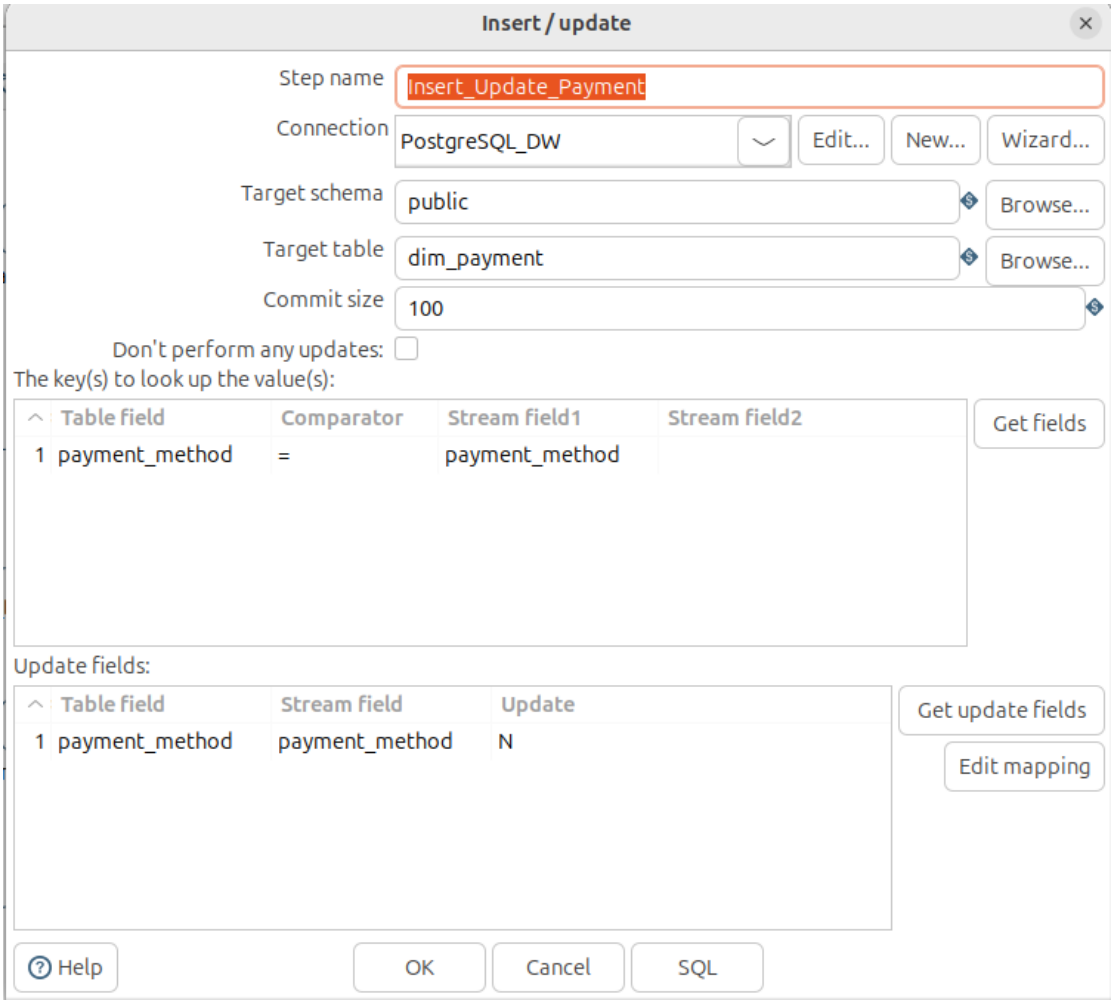
Bu adım, dim_date tablosuna yazan Insert/Update bileşenidir. Anahtar (key) olarak full_date seçilmiştir: gelen her tarih tabloda aranır, aynı tarih daha önce eklenmişse (örneğin o gün içinde birden fazla sipariş verilmişse) satır güncellenir, yoksa yeni bir tarih kaydı olarak eklenir. Bu sayede dim_date tablosunda her takvim günü için yalnızca tek bir satır bulunur; 500 sipariş, 278 farklı benzersiz tarihe indirgenmiş olur.

12.8. Adım:



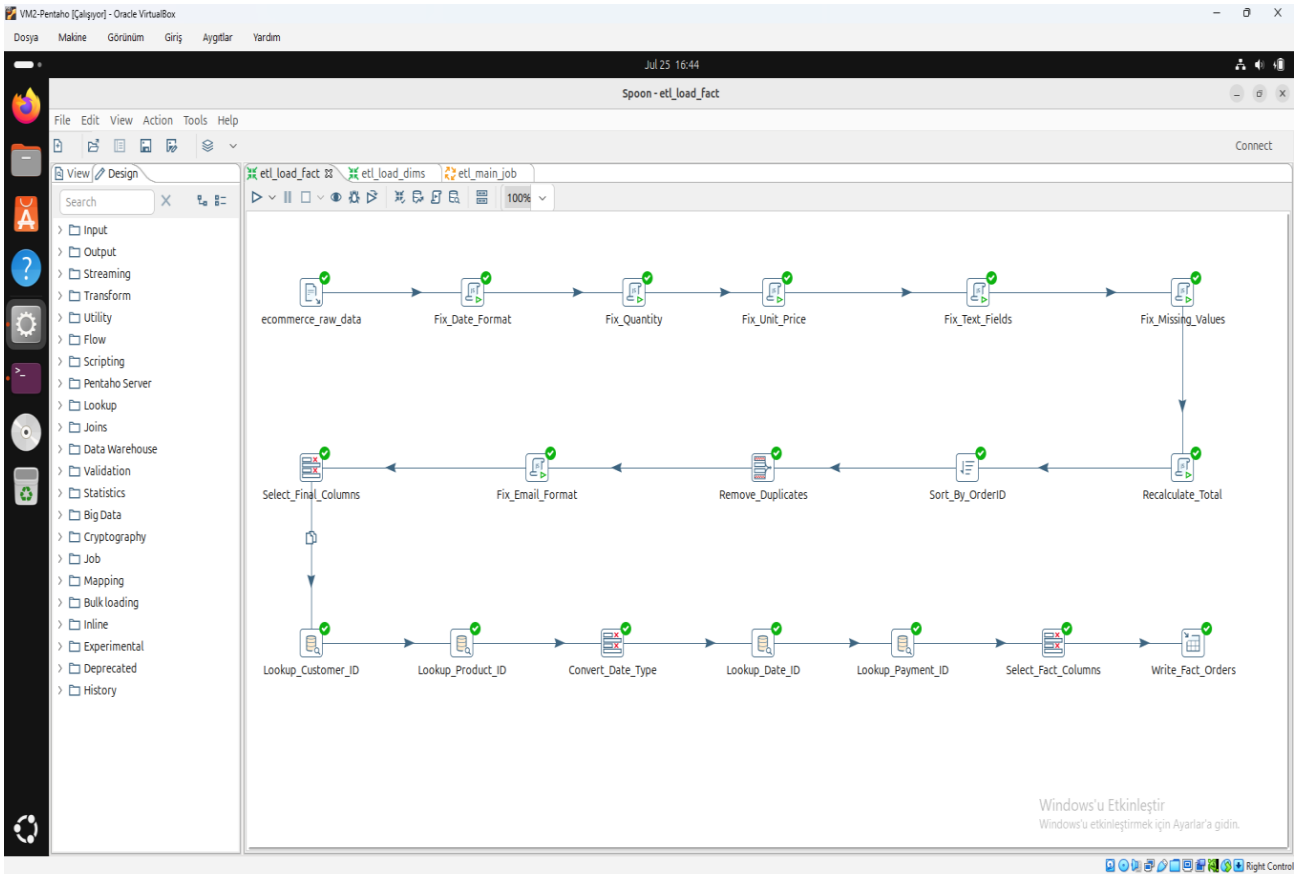
Bu adım, temiz veri setinden yalnızca payment_method alanını seçen bir Select Values bileşenidir. Diğer tüm alanlar bu kolda elenir; çünkü bu kolun tek görevi dim_payment tablosuna yazılacak sade veriyi hazırlamaktır.

12.9. Adım:



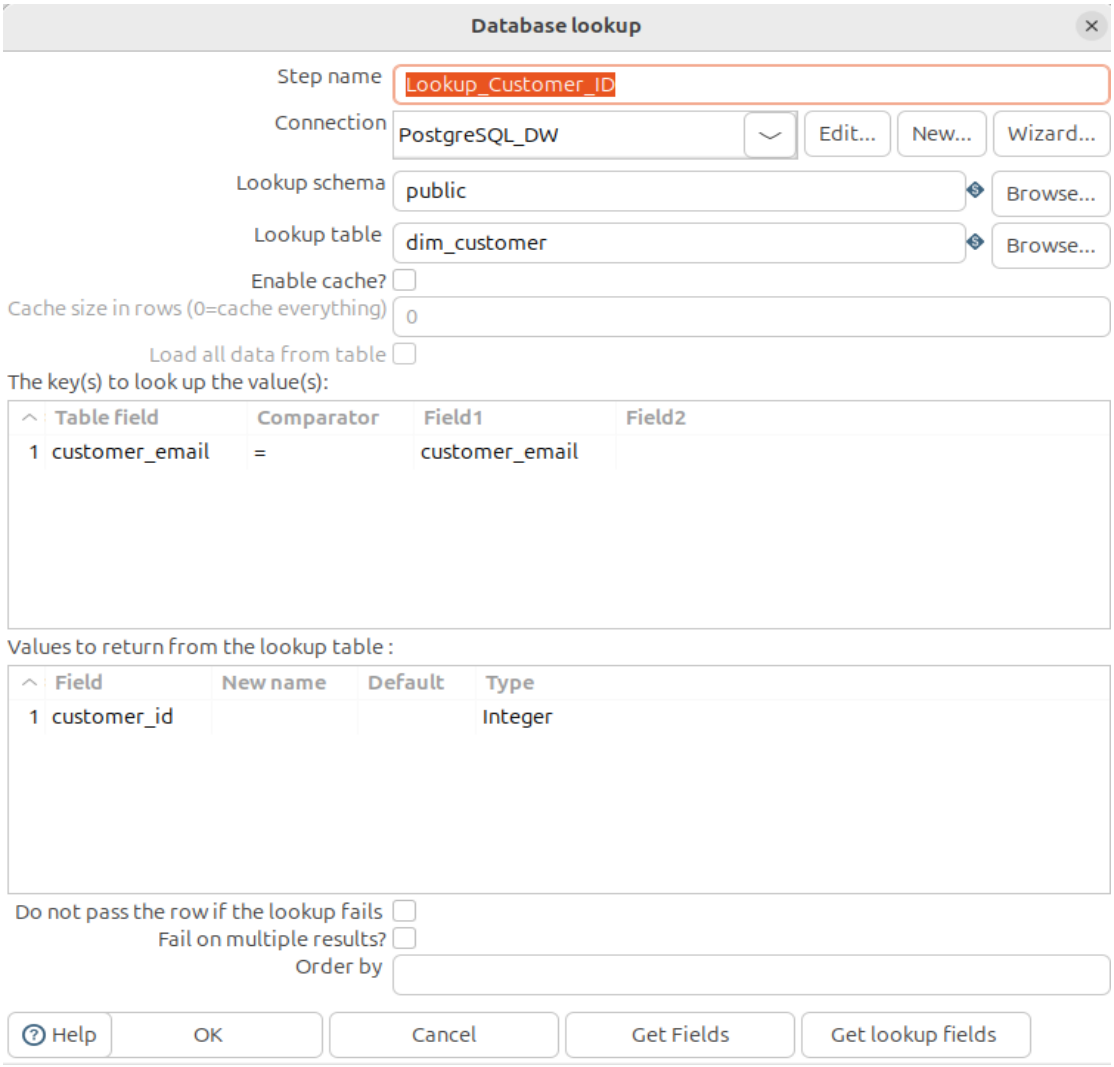
Bu adım, dim_payment tablosuna yazan Insert/Update bileşenidir. Anahtar (key) olarak payment_method seçilmiştir: gelen her ödeme yöntemi tabloda aranır, aynı yöntem daha önce eklenmişse satır güncellenir, yoksa yeni bir kayıt olarak eklenir. Bu sayede farklı siparişlerdeki aynı ödeme yöntemleri (örneğin "Kredi Kartı") dim_payment tablosunda tek bir satırda toplanır; 500 sipariş, yalnızca 5 farklı ödeme yöntemine (4 gerçek yöntem + eksik veriler için "Belirtilmemiş") indirgenmiş olur.

İki Tranformation dosyamızdan etl_load_dims bitti. Şimdi sırada etl_load_fact var.



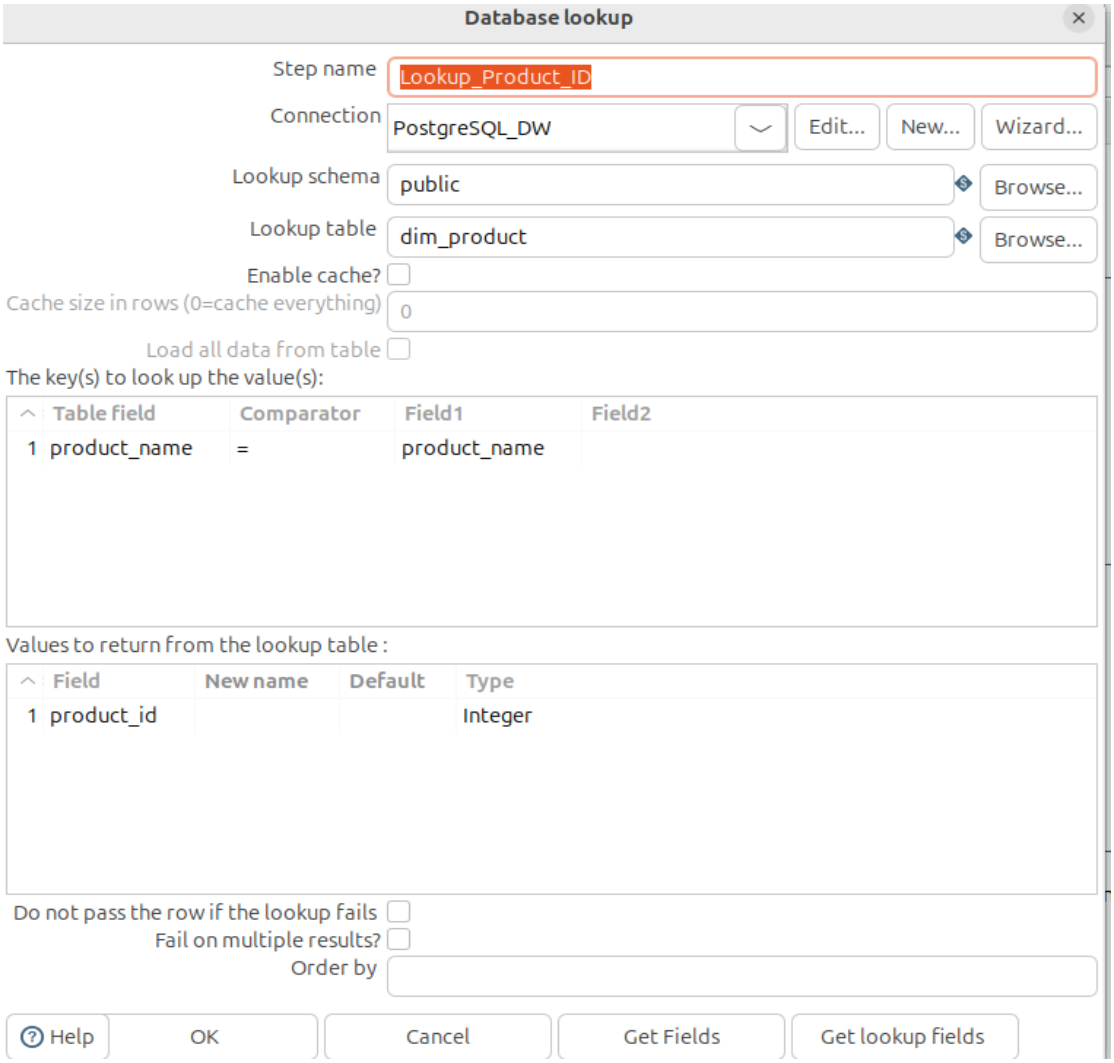
Bu transformation, ecommerce_raw_data'dan Select_Final_Columns'a kadar olan adımların tamamını (Fix_Date_Format, Fix_Quantity, Fix_Unit_Price, Fix_Text_Fields, Fix_Missing_Values, Recalculate_Total, Sort_By_OrderID, Remove_Duplicates, Fix_Email_Format, Select_Final_Columns) etl_load_dims ile birebir aynı şekilde tekrarlar; çünkü fact tablosuna yazılacak veri de aynı temiz veri setinden türetilmelidir. Bu yüzden burada bu adımları tekrar anlatmak yerine, doğrudan Select_Final_Columns sonrasında başlayan ve fact tablosuna özgü yeni adımlara odaklanacağız.

1. Adım:



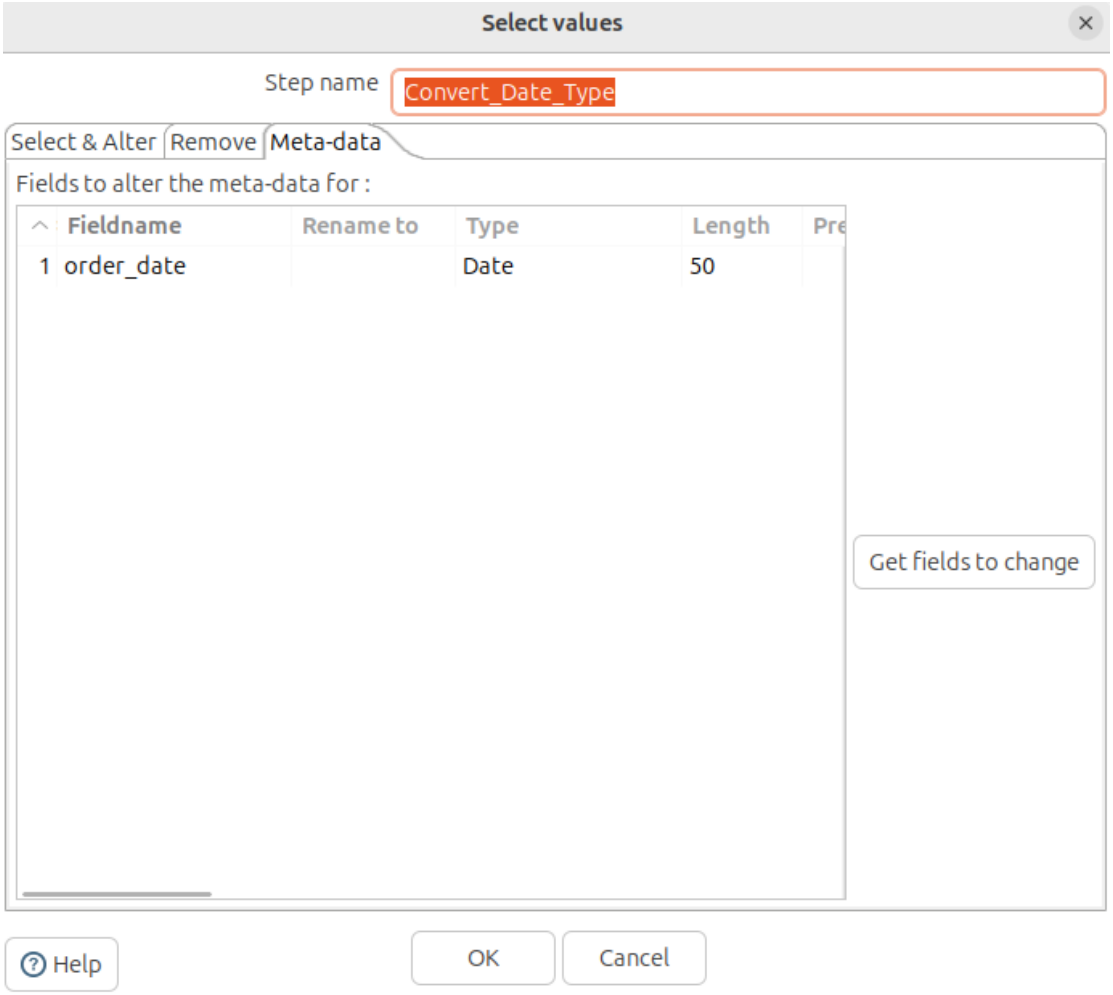
Bu adım, temiz veri setindeki customer_email alanını dim_customer tablosunda arayan bir Database Lookup bileşenidir. Bir önceki adımda dim_customer tablosu zaten doldurulmuş olduğu için, burada her satırın e-posta adresi tabloda aranır ve eşleşen müşterinin customer_id değeri satıra eklenir. Bu sayede fact_orders tablosuna müşterinin tüm bilgileri tekrar yazılmaz, sadece bir customer_id foreign key'i olarak taşınır.

2. Adım:



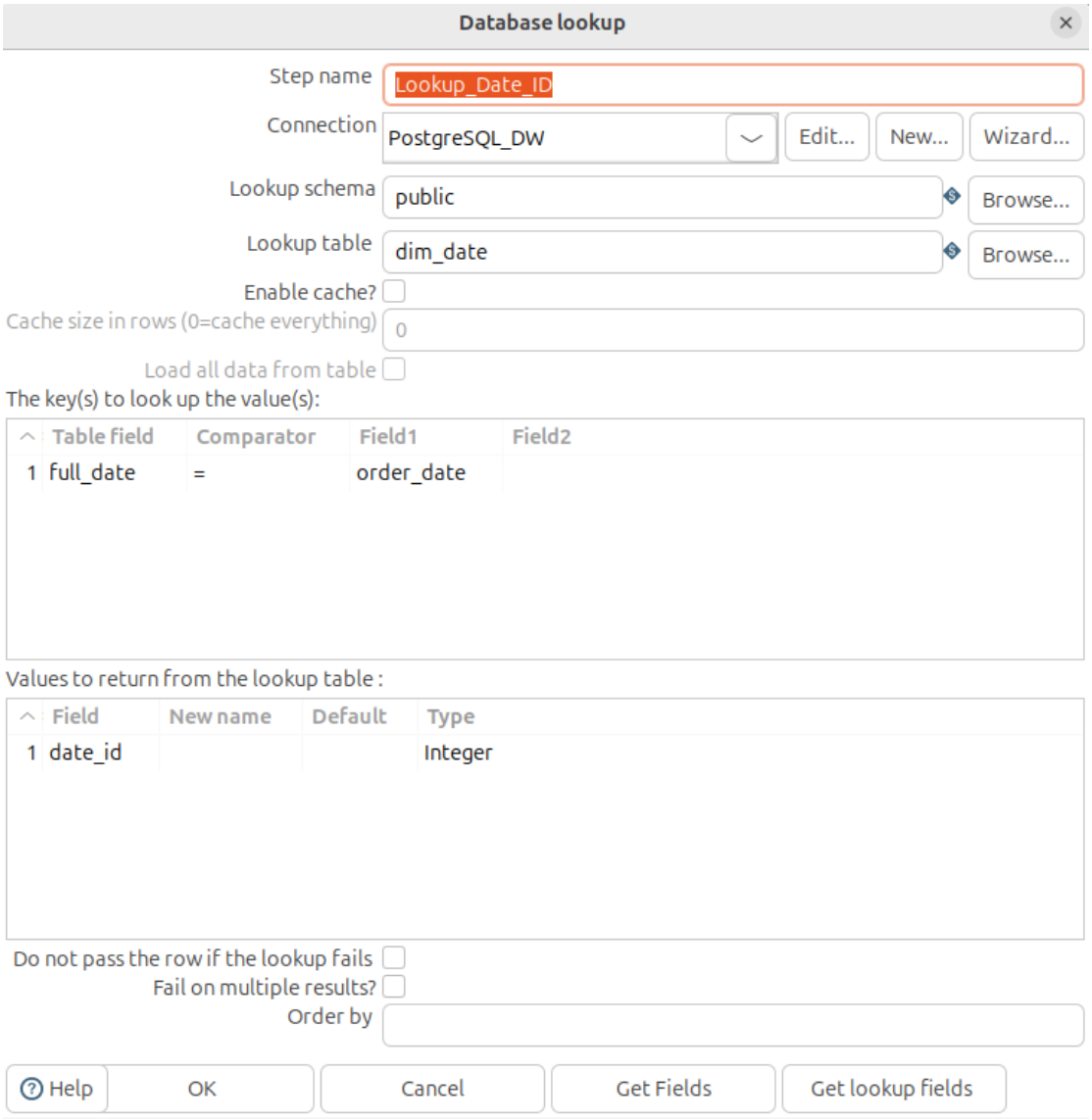
Bu adım, product_name alanını dim_product tablosunda arayan bir Database Lookup bileşenidir. Anahtar olarak yalnızca product_name kullanılması kritik bir detaydır: daha önce bahsettiğimiz gibi, bu arama başlangıçta product_name + product_category ikilisiyle yapıldığında, kategorisi eksik ya da farklı yazılmış 10 sipariş için eşleşme bulunamıyor ve product_id boş (NULL) kalıyordu. Anahtarın yalnızca product_name'e indirgenmesiyle bu sorun giderildi ve tüm siparişler doğru product_id ile eşleşti.

3. Adım:



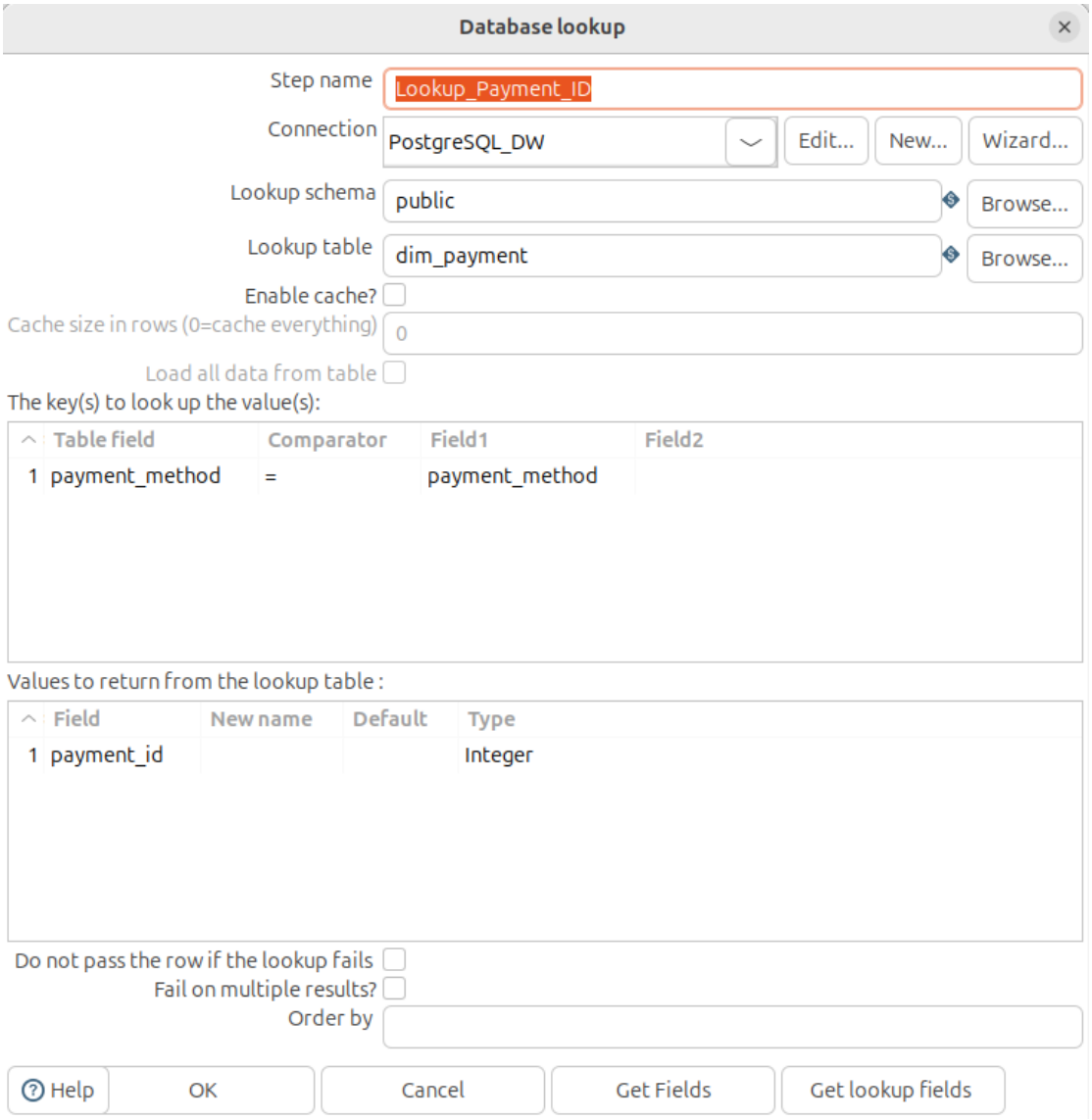
Bu adım, order_date alanının veri tipini String'den Date'e çeviren bir Select Values (Meta-data) bileşenidir. Buraya kadar order_date alanı, standartlaştırılmış olsa da hâlâ metin (String) olarak tutuluyordu; ancak bir sonraki adımda dim_date tablosundaki full_date alanıyla (Date tipinde) eşleştirme yapılabilmesi için, iki tarafın da aynı veri tipinde olması gerekir. Bu adım, order_date'i gerçek bir tarih tipine dönüştürerek bu eşleşmeyi mümkün kılar.

4. Adım:



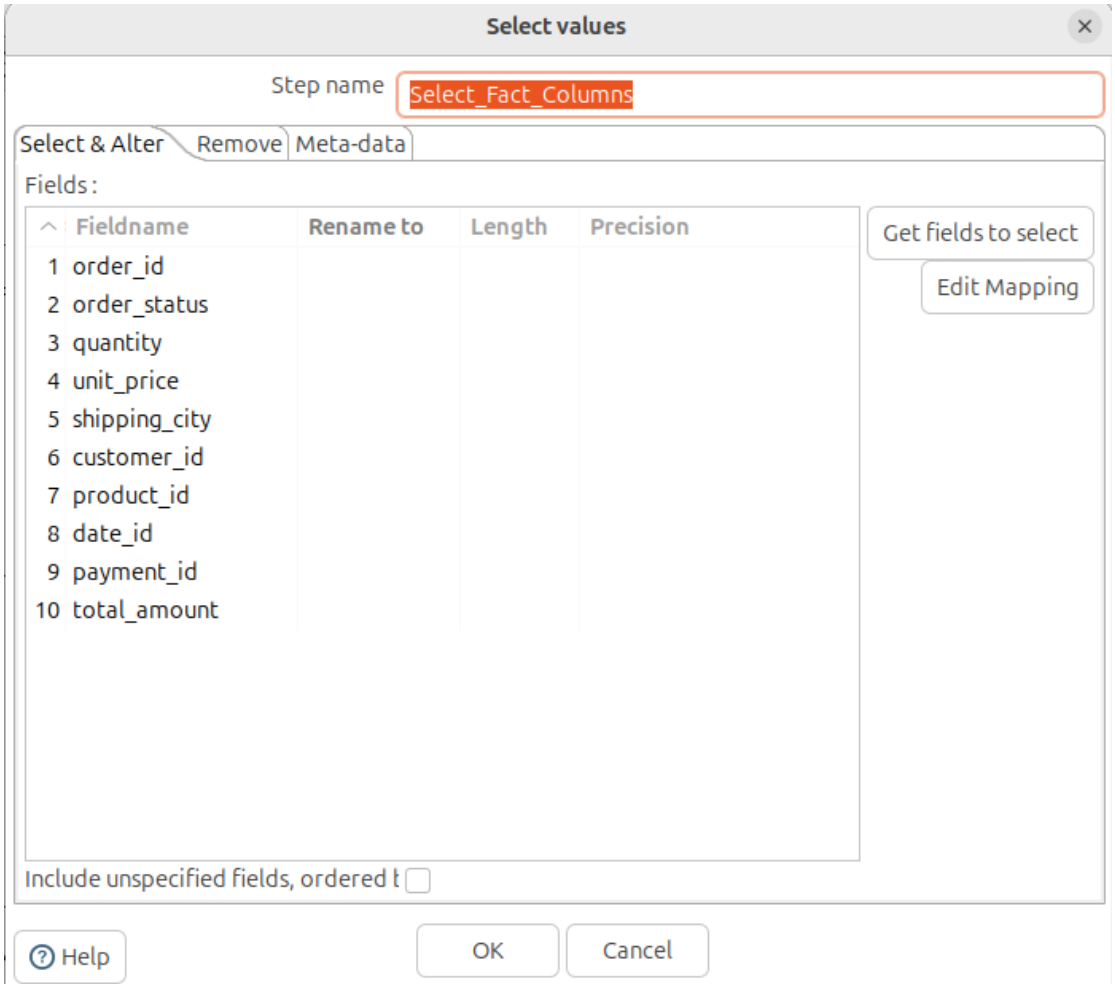
Bu adım, bir önceki adımda Date tipine dönüştürülen order_date alanını dim_date tablosundaki full_date alanıyla eşleştiren bir Database Lookup bileşenidir. Eşleşme bulunan satıra, o tarihe karşılık gelen date_id değeri eklenir. Bu adımın sorunsuz çalışabilmesi, tam olarak bir önceki adımda yapılan tip dönüşümüne bağlıydı.

5. Adım:



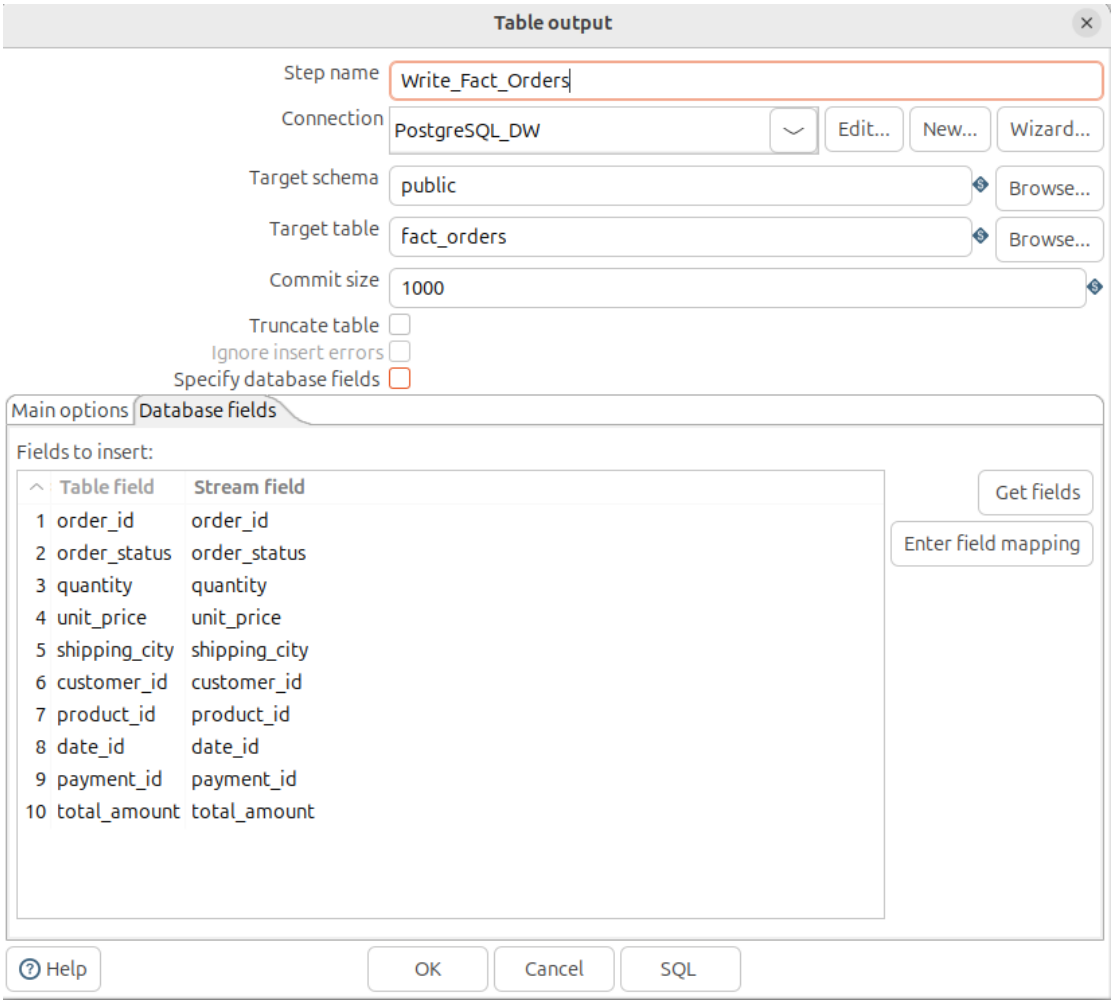
Bu adım, payment_method alanını dim_payment tablosunda arayan bir Database Lookup bileşenidir. Diğer üç lookup adımıyla aynı mantıkla çalışır: eşleşen ödeme yönteminin payment_id değeri satıra eklenir. Bu adımın tamamlanmasıyla birlikte, fact tablosuna yazılacak her satır artık dört boyutun da (müşteri, ürün, tarih, ödeme) kimlik numaralarını taşır hale gelmiştir.

6. Adım:



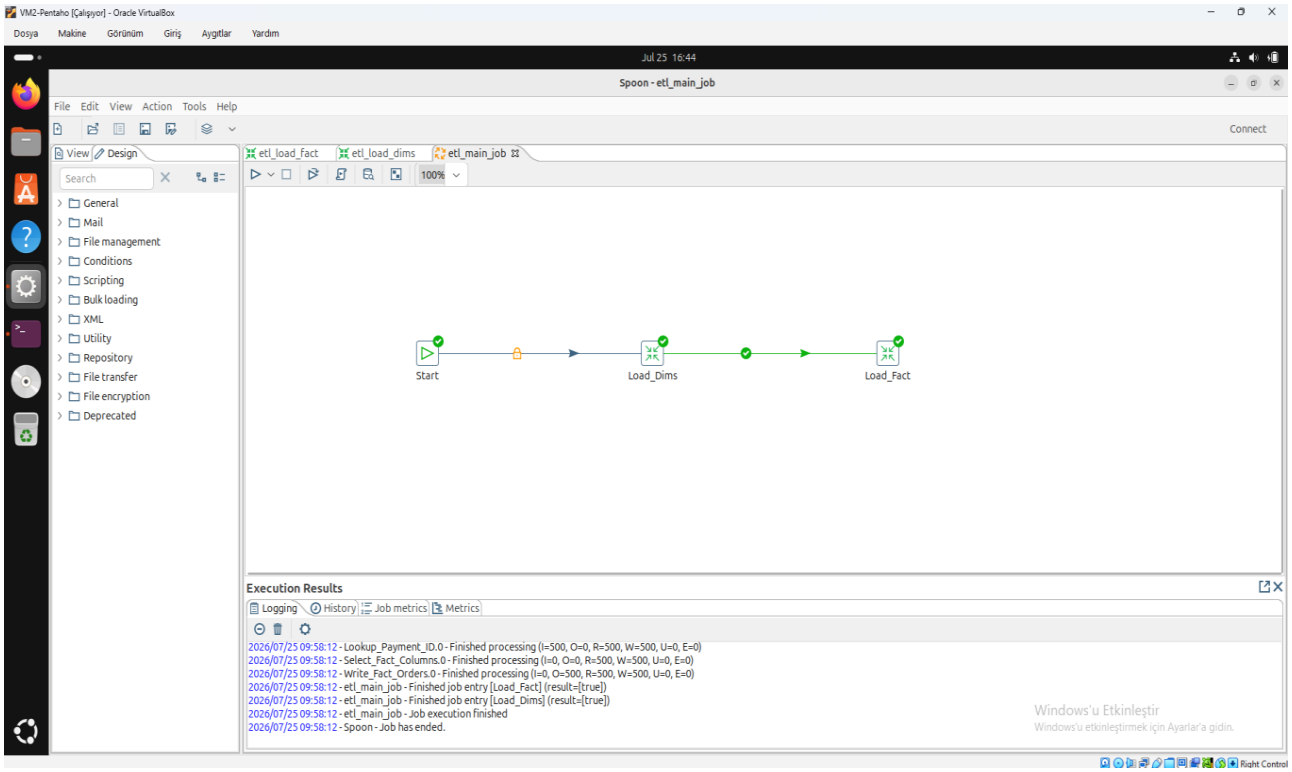
Bu adım, dört lookup adımından sonra oluşan geniş alan setinden yalnızca fact_orders tablosuna yazılacak 10 sütunu (order_id, order_status, quantity, unit_price, shipping_city, customer_id, product_id, date_id, payment_id, total_amount) seçen bir Select Values bileşenidir. Müşteri adı, ürün adı, ödeme yöntemi metni gibi artık gereksiz hale gelen alanlar (çünkü bunlar yerlerini foreign key'lere bıraktı) burada akıştan düşürülür. Bu sayede fact tablosuna, yıldız şemasının gerektirdiği sade ve normalize edilmiş veri seti iletilmiş olur.

7. Adım:



Bu adım, temizlenmiş ve dört foreign key ile zenginleştirilmiş son veriyi fact_orders tablosuna yazan bir Table Output bileşenidir. Dim tablolarındaki Insert/Update adımlarının aksine, burada doğrudan bir ekleme (insert) işlemi yapılır; çünkü her sipariş (order_id) benzersizdir ve zaten bir önceki transformation'da (Remove_Duplicates adımıyla) mükerrer kayıtlar elenmiştir, güncelleme mantığına ihtiyaç yoktur. Fields to insert tablosunda, 10 sütunun (order_id, order_status, quantity, unit_price, shipping_city, customer_id, product_id, date_id, payment_id, total_amount) her biri, akıştaki karşılığıyla (Stream field) birebir eşleştirilmiştir.

etl_load_dims ve etl_load_fact dosyalarımızı inceledik. Şimdi sırada ikisini birleştiren etl_main_job dosyamız var.



etl_load_dims ve etl_load_fact, aslında birbirinden bağımsız iki ayrı transformation dosyasıdır ve aralarında doğrudan bir bağlantı yoktur. Ancak etl_load_fact'in doğru çalışabilmesi için, dört boyut tablosunun (dim_customer, dim_product, dim_date, dim_payment) etl_load_dims tarafından tamamen doldurulmuş olması gerekir; aksi halde Lookup adımları eşleşme bulamaz.

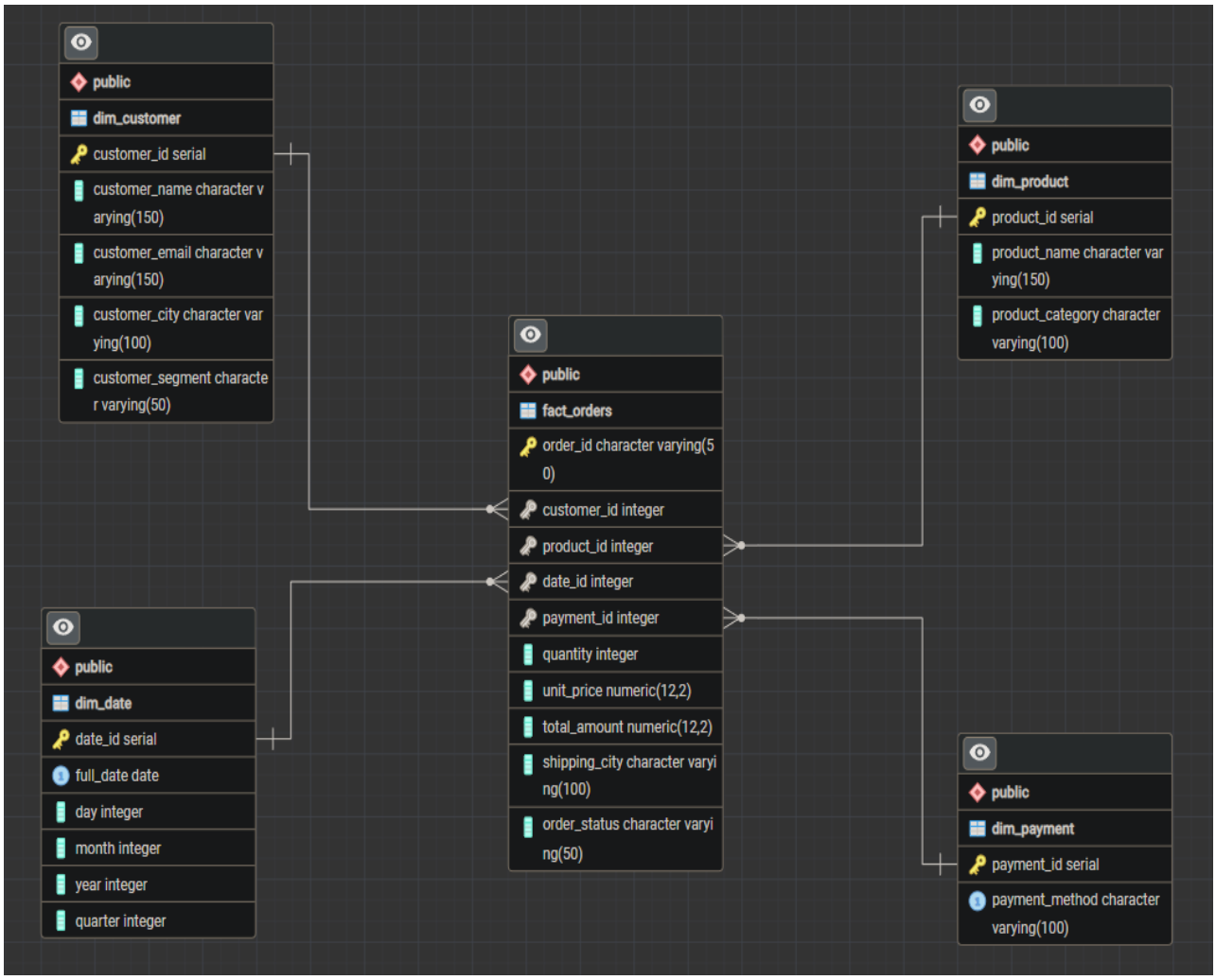
Bu bağımlılığı yönetmek için Pentaho'da bir üst yapı olan Job (.kjb) kullanıldı. etl_main_job, üç adımdan oluşan basit ama kritik bir sıralı akıştır: Start → Load_Dims → Load_Fact. Job yapısı, transformation'ların aksine adımları birbirini bekleyerek çalıştırır; yani Load_Dims tamamlanmadan Load_Fact hiçbir şekilde başlamaz.

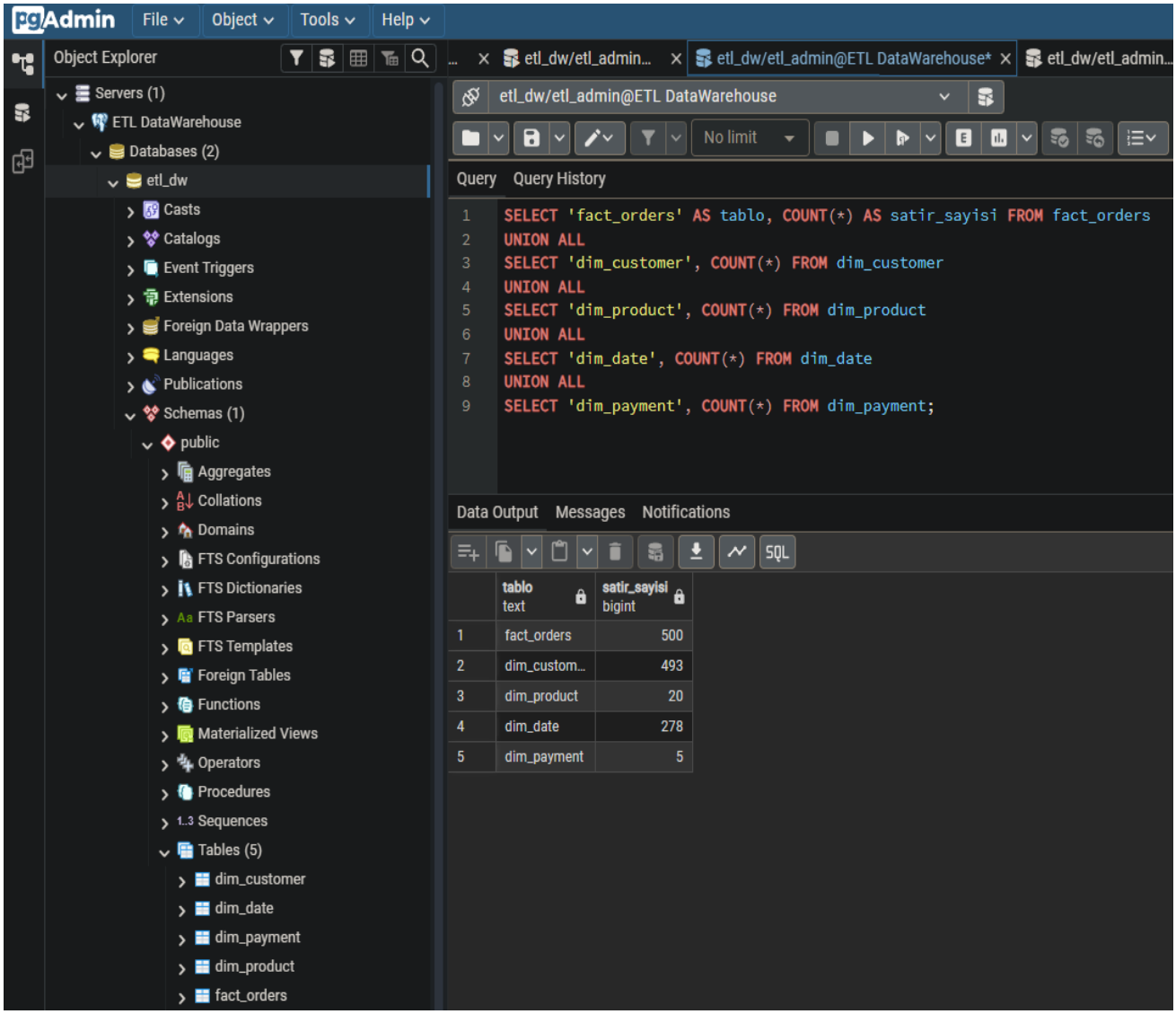
Bu yapı, race condition sorununun kalıcı çözümüydü. İlk denemede tüm adımlar tek bir transformation içinde paralel kollar halinde çalıştığında, boyut tabloları henüz dolmadan fact tarafındaki arama işlemleri başlıyor ve foreign key'lerin yaklaşık %99,8'i NULL çıkıyordu. Akışın Load_Dims ve Load_Fact olarak iki ayrı transformation'a bölünüp bir Job içinde sıralı bağlanması, bu yarışı ortadan kaldırdı.

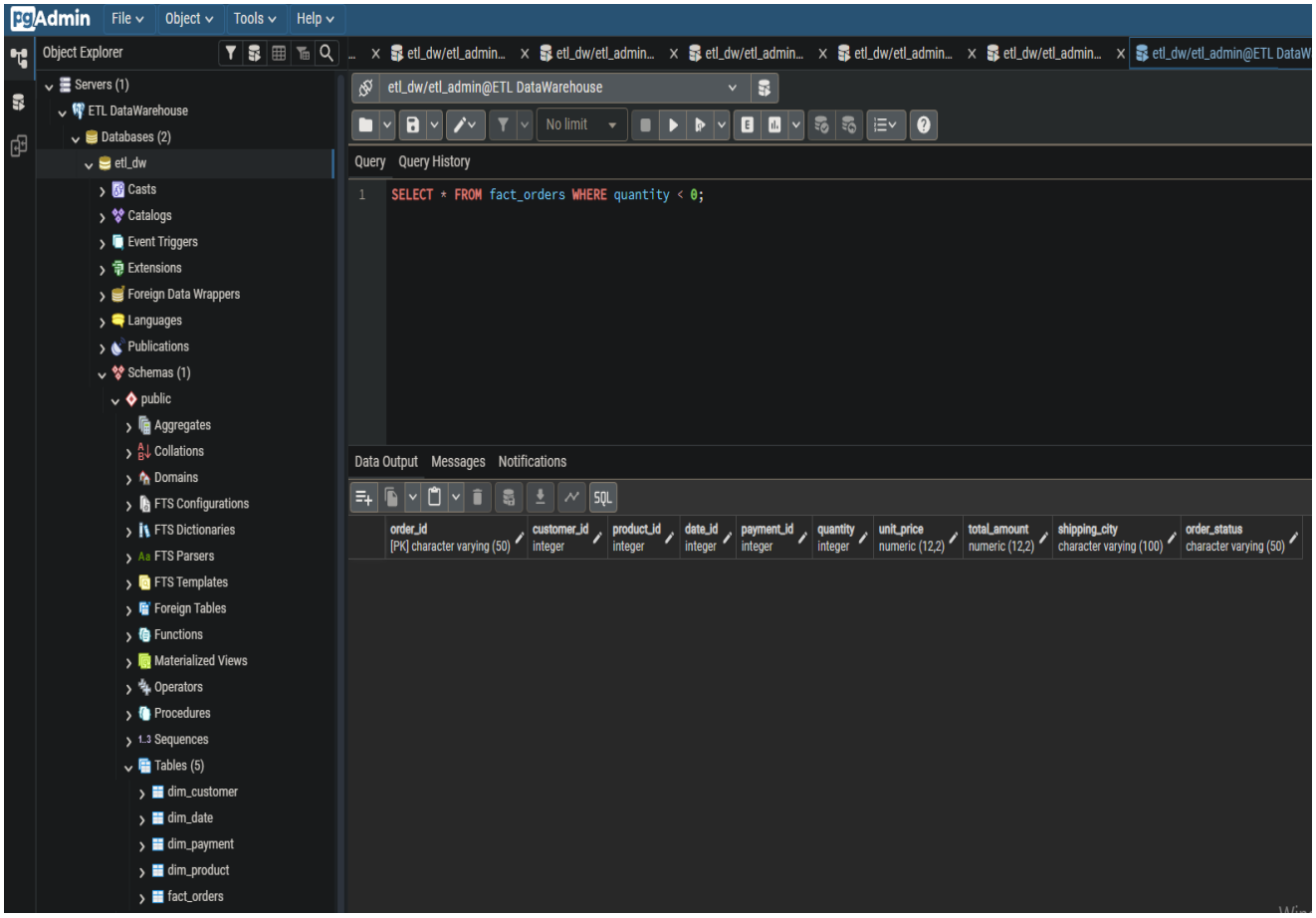
Bu adımla birlikte Transformation ana başlığı tamamlanmış oldu. Ham veri, artık VM1'deki PostgreSQL veri ambarında, sıfır hatayla yapılandırılmış bir yıldız şeması olarak duruyor.

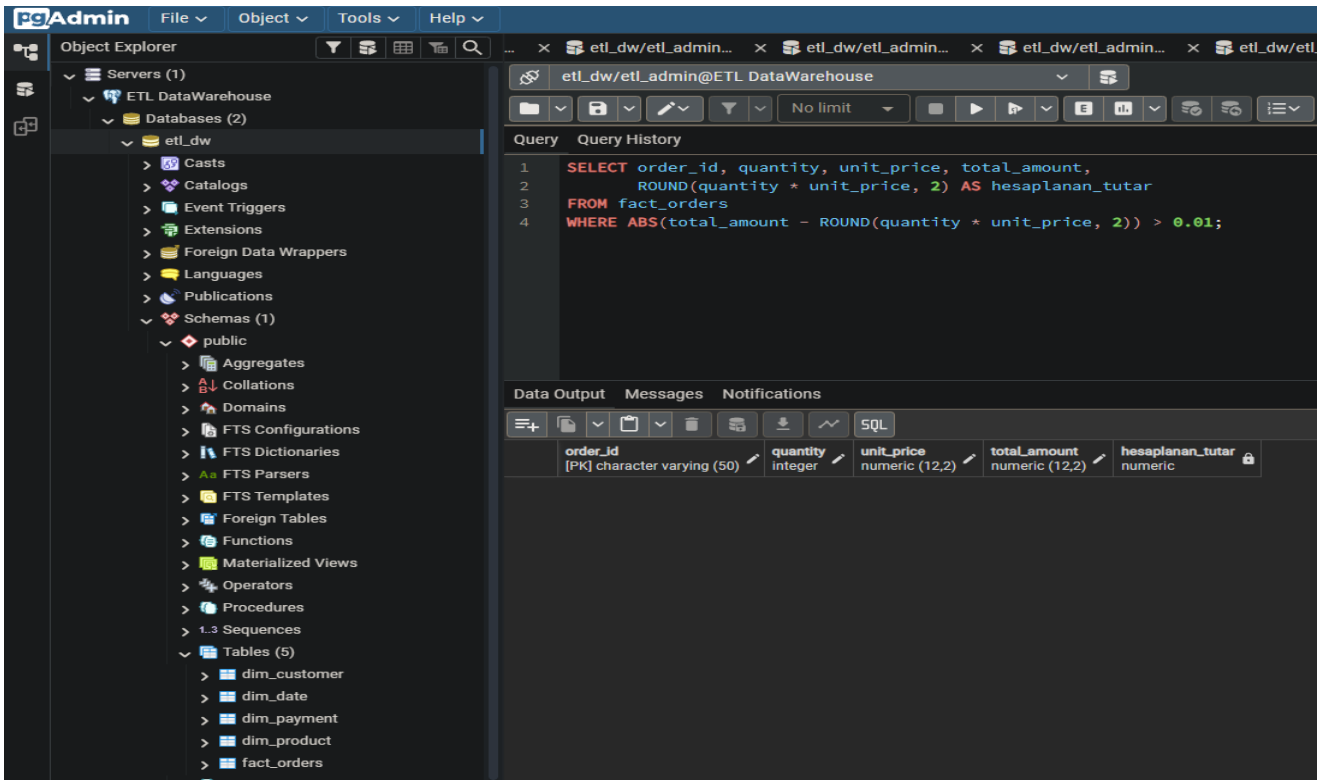
LOAD:

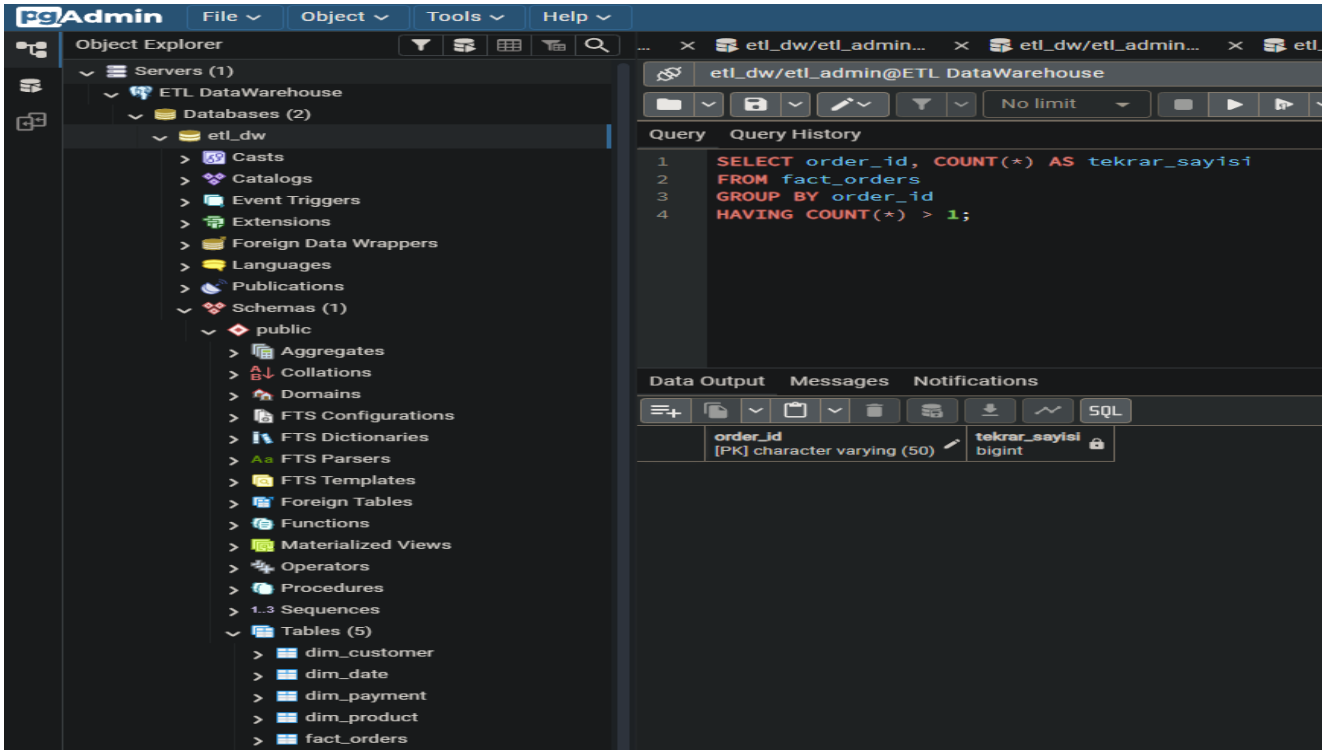
Gerçek hayatta Load, ETL sürecinin son adımıdır ve Transform aşamasında temizlenmiş, dönüştürülmüş verinin, daha önce tasarlanmış hedef şemaya kalıcı olarak yazılmasıdır. Bu aşamada asıl önemli olan, verinin sadece "yazılmış" olması değil, aynı zamanda doğrulanmış olmasıdır; çünkü Transform aşamasında ne kadar dikkatli çalışılırsa çalışılsın, öngörülemeyen bir kenar durum (edge case) verinin bir kısmının eksik ya da hatalı yüklenmesine neden olabilir. Bu yüzden gerçek projelerde Load adımının hemen ardından, satır sayıları, foreign key bütünlüğü ve toplam tutarlar gibi kontrollerin yapıldığı bir doğrulama (validation) aşaması gelir.
Bizim projemizde bu adım, etl_main_job'ın Load_Dims ve Load_Fact adımlarının başarıyla tamamlanmasıyla fiilen gerçekleşmiş oldu; veri, VM1 üzerindeki PostgreSQL veritabanına (etl_dw), önceden SQL ile kurduğumuz yıldız şemasına yazıldı. Ancak yazma işleminin başarılı görünmesi tek başına yeterli değildi; bu yüzden pgAdmin üzerinden Query Tool'a geçilerek bir dizi doğrulama sorgusu çalıştırıldı. Ayrıca kurduğumuz bu yıldız şeması görsel olarak da incelendi.

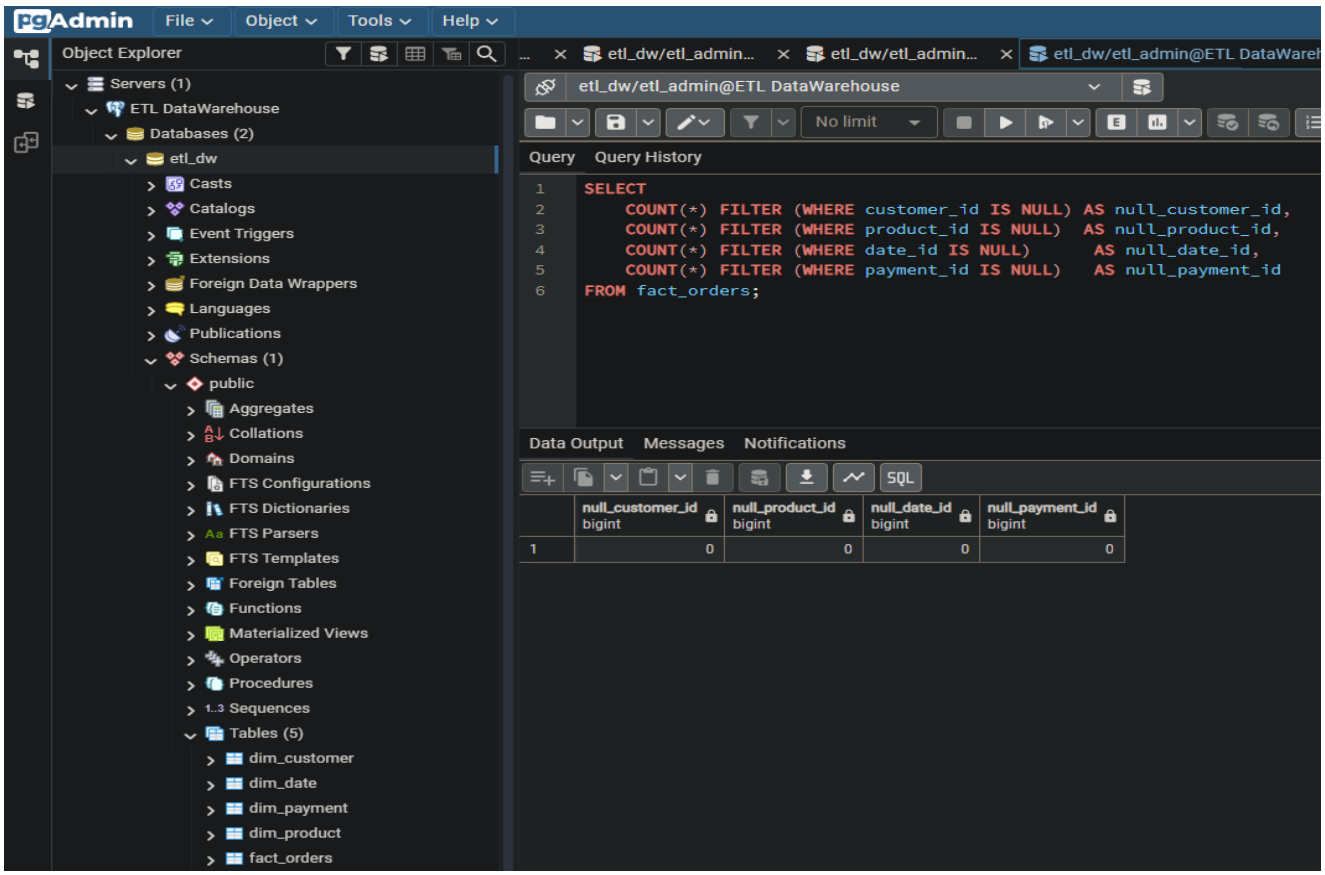


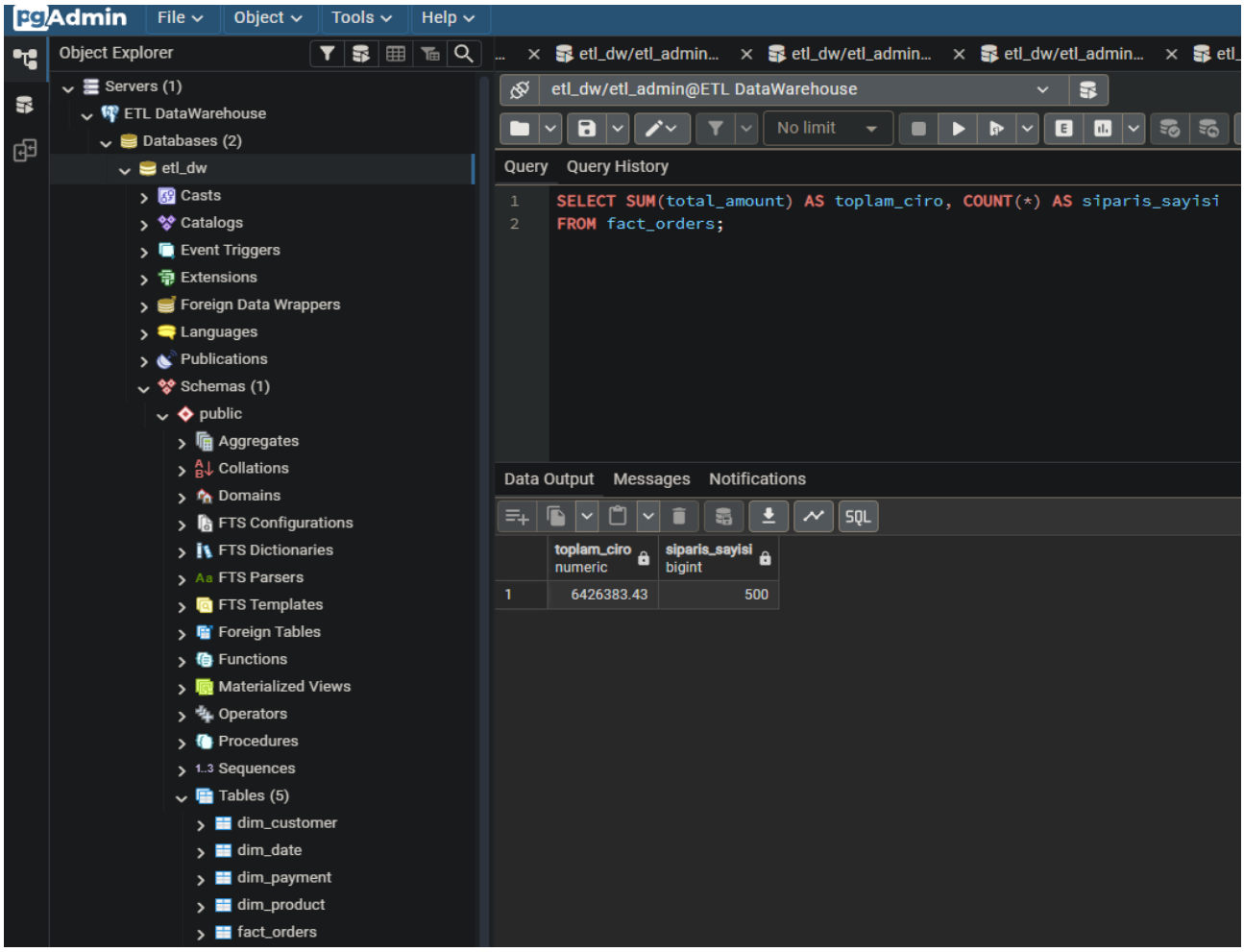

pgAdmin
File
Object
Tools
Help
Object Explorer
etl_dw/etl_admin...
etl_dw/etl_admin@ETL DataWarehouse*
etl_dw/etl_admin...
Servers (1)
ETL DataWarehouse
Databases (2)
etl_dw
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (5)
dim_customer
dim_date
dim_payment
dim_product
fact_orders
etl_dw/etl_admin@ETL DataWarehouse
No limit
Query
Query History
SELECT 'fact_orders' AS tablo, COUNT(*) AS satir_sayisi FROM fact_orders
UNION ALL
SELECT 'dim_customer', COUNT(*) FROM dim_customer
UNION ALL
SELECT 'dim_product', COUNT(*) FROM dim_product
UNION ALL
SELECT 'dim_date', COUNT(*) FROM dim_date
UNION ALL
SELECT 'dim_payment', COUNT(*) FROM dim_payment;
Data Output
Messages
Notifications
SQL
tablo text
satir_sayisi bigint
1 fact_orders 500
2 dim_custom... 493
3 dim_product 20
4 dim_date 278
5 dim_payment 5


pgAdmin
File
Object
Tools
Help
Object Explorer
Servers (1)
ETL DataWarehouse
Databases (2)
etl_dw
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (5)
dim_customer
dim_date
dim_payment
dim_product
fact_orders
etl_dw/etl_admin@ETL DataWarehouse
No limit
Query
Query History
SELECT * FROM fact_orders WHERE quantity < 0;
Data Output
Messages
Notifications
SQL
order_id
[PK] character varying (50)
customer_id
integer
product_id
integer
date_id
integer
payment_id
integer
quantity
integer
unit_price
numeric (12,2)
total_amount
numeric (12,2)
shipping_city
character varying (100)
order_status
character varying (50)


pgAdmin
File
Object
Tools
Help
Object Explorer
Servers (1)
ETL DataWarehouse
Databases (2)
etl_dw
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (5)
dim_customer
dim_date
dim_payment
dim_product
fact_orders
etl_dw/etl_admin@ETL DataWarehouse
No limit
Query
Query History
SELECT order_id, quantity, unit_price, total_amount,
ROUND(quantity * unit_price, 2) AS hesaplanan_tutar
FROM fact_orders
WHERE ABS(total_amount - ROUND(quantity * unit_price, 2)) > 0.01;
Data Output
Messages
Notifications
SQL
order_id
[PK] character varying (50)
quantity
integer
unit_price
numeric (12,2)
total_amount
numeric (12,2)
hesaplanan_tutar
numeric


pgAdmin
File
Object
Tools
Help
Object Explorer
Servers (1)
ETL DataWarehouse
Databases (2)
etl_dw
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (5)
dim_customer
dim_date
dim_payment
dim_product
fact_orders
etl_dw/etl_admin...
etl_dw/etl_admin...
etl_dw/etl_admin@ETL DataWarehouse
No limit
Query
Query History
1 SELECT order_id, COUNT(*) AS tekrar_sayisi
2 FROM fact_orders
3 GROUP BY order_id
4 HAVING COUNT(*) > 1;
Data Output
Messages
Notifications
SQL
order_id
[PK] character varying (50)
tekrar_sayisi
bigint


pgAdmin
File
Object
Tools
Help
Object Explorer
Servers (1)
ETL DataWarehouse
Databases (2)
etl_dw
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (5)
dim_customer
dim_date
dim_payment
dim_product
fact_orders
etl_dw/etl_admin...
etl_dw/etl_admin...
etl_dw/etl_admin@ETL DataWarehouse
No limit
Query
Query History
1 SELECT
2 COUNT(*) FILTER (WHERE customer_id IS NULL) AS null_customer_id,
3 COUNT(*) FILTER (WHERE product_id IS NULL) AS null_product_id,
4 COUNT(*) FILTER (WHERE date_id IS NULL) AS null_date_id,
5 COUNT(*) FILTER (WHERE payment_id IS NULL) AS null_payment_id
6 FROM fact_orders;
Data Output
Messages
Notifications
SQL
null_customer_id
bigint
null_product_id
bigint
null_date_id
bigint
null_payment_id
bigint
1
0
0
0
0



Bu doğrulama sürecinde altı farklı SQL sorgusu çalıştırıldı. İlk sorgu, beş tablonun da satır sayısını tek bir sonuç kümesinde listeleyerek beklenen rakamlarla (500 fact, 493 müşteri, 20 ürün, 278 tarih, 5 ödeme yöntemi) örtüşüp örtüşmediğini kontrol etti. İkinci sorgu, fact_orders tablosundaki dört foreign key alanında (customer_id, product_id, date_id, payment_id) hiç NULL değer kalıp kalmadığını, yani daha önce bahsettiğimiz race condition sorununun tamamen giderilip giderilmediğini teyit etti. Üçüncü sorgu, order_id alanına göre gruplama yaparak birden fazla kez görünen bir sipariş olup olmadığını, yani mükerrer kayıt bulunup bulunmadığını kontrol etti. Dördüncü sorgu, her satırda total_amount değerinin quantity × unit_price çarpımıyla birebir tutarlı olup olmadığını karşılaştırarak, Transform aşamasındaki Recalculate_Total adımının doğru çalıştığını doğruladı. Beşinci sorgu, negatif adet (quantity) içeren herhangi bir satırın kalıp kalmadığını kontrol ederek Fix_Quantity adımının etkinliğini teyit etti. Son olarak altıncı sorgu, tüm siparişlerin toplam cirosunu ve sipariş sayısını hesaplayarak, bu rakamların ileride BI aşamasında kuracağımız dashboard'daki KPI değerleriyle birebir örtüşüp örtüşmediğinin karşılaştırılabilmesini sağladı. Bu altı sorgunun sonucu, veri ambarının hem yapısal hem de sayısal olarak tamamen güvenilir olduğunu ortaya koydu.

Ham verinin doğuşundan, temizlenip güvenilir bir veri ambarına yüklenmesine kadar geçen ETL sürecimiz böylece tamamlanmış oldu. Artık elimizde, yöneticinin sorusuna cevap verebilecek sağlam bir veri temeli var. Şimdi sırada bu temeli görselleştirip yorumlayacağımız iş zekası (BI) aşaması var.

BI - Business Intelligence:

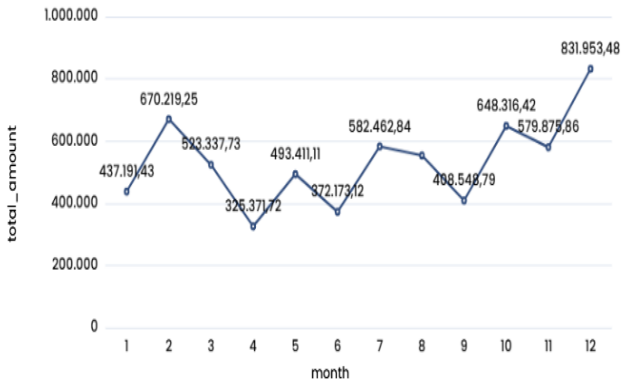
Gerçek hayatta İş Zekası (Business Intelligence - BI), veri ambarındaki ham sayıların, karar vericilerin (yöneticilerin) anlayabileceği görsel ve yorumlanabilir bir forma dönüştürülmesi sürecidir. Buraya kadarki tüm aşamalar (Extract, Transform, Load) teknik ekipler için anlamlıydı; ancak bir yönetici için 500 satırlık bir SQL tablosunun hiçbir pratik değeri yoktur. BI aşamasının amacı, tam olarak bu uçurumu kapatmaktır: karmaşık veriyi, tek bakışta anlaşılabilecek grafikler, tablolar ve özet göstergeler (KPI'lar) haline getirmek.

Gerçek dünyada bu iş için Power BI, Tableau, Looker gibi araçlar kullanılır. Bu araçların ortak mantığı şudur: veri ambarındaki tablolar arasına (genelde bir yıldız şeması üzerinden) ilişkiler kurulur, ardından bu ilişkili veri modeli üzerine sürükle-bırak mantığıyla grafikler, tablolar ve filtreler eklenerek bir dashboard oluşturulur. Bizim projemizde bu amaçla Data Plato aracı kullanıldı. VM1'den dışa aktarılan 5 tablo (fact_orders, dim_customer, dim_product, dim_date, dim_payment) Data Plato'ya yüklenerek, veri ambarındaki yıldız şeması burada da (fact_orders merkeze alınıp dört boyutla Left Join ilişkileri kurularak) yeniden inşa edildi. Bu veri modeli üzerine, yöneticinin en başta sorduğu "nerede ne kadar kazanıyorum, hangi ürünlerden çok kazanıyorum" sorularına cevap verecek 6 sekmelik bir dashboard oluşturuldu: KPI, Genel Bakış, Şehirler, Harita, Ürünler ve Şehire Göre Kategoriler.



Aylık Ciro Trendi (2025)

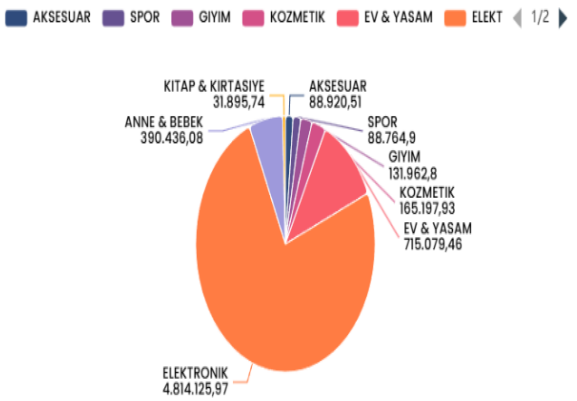
Enter Description



Enter Foot Note

Ürün Kategorisine Göre Satış Dağılımı

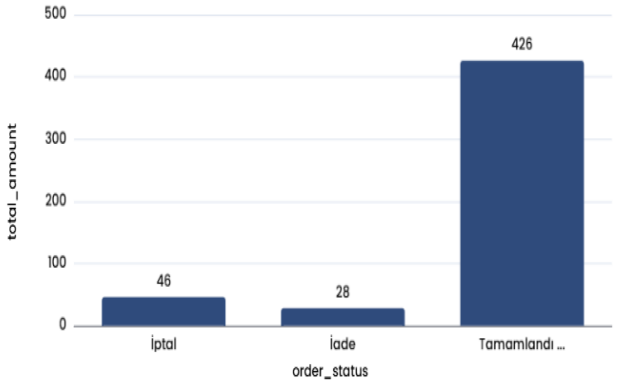
Enter Description



Enter Foot Note

Sipariş Durumu Dağılımı

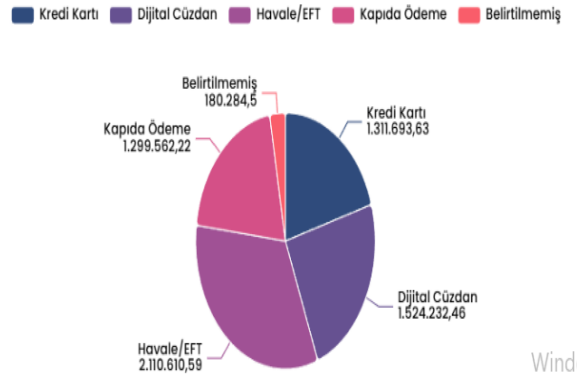
Enter Description



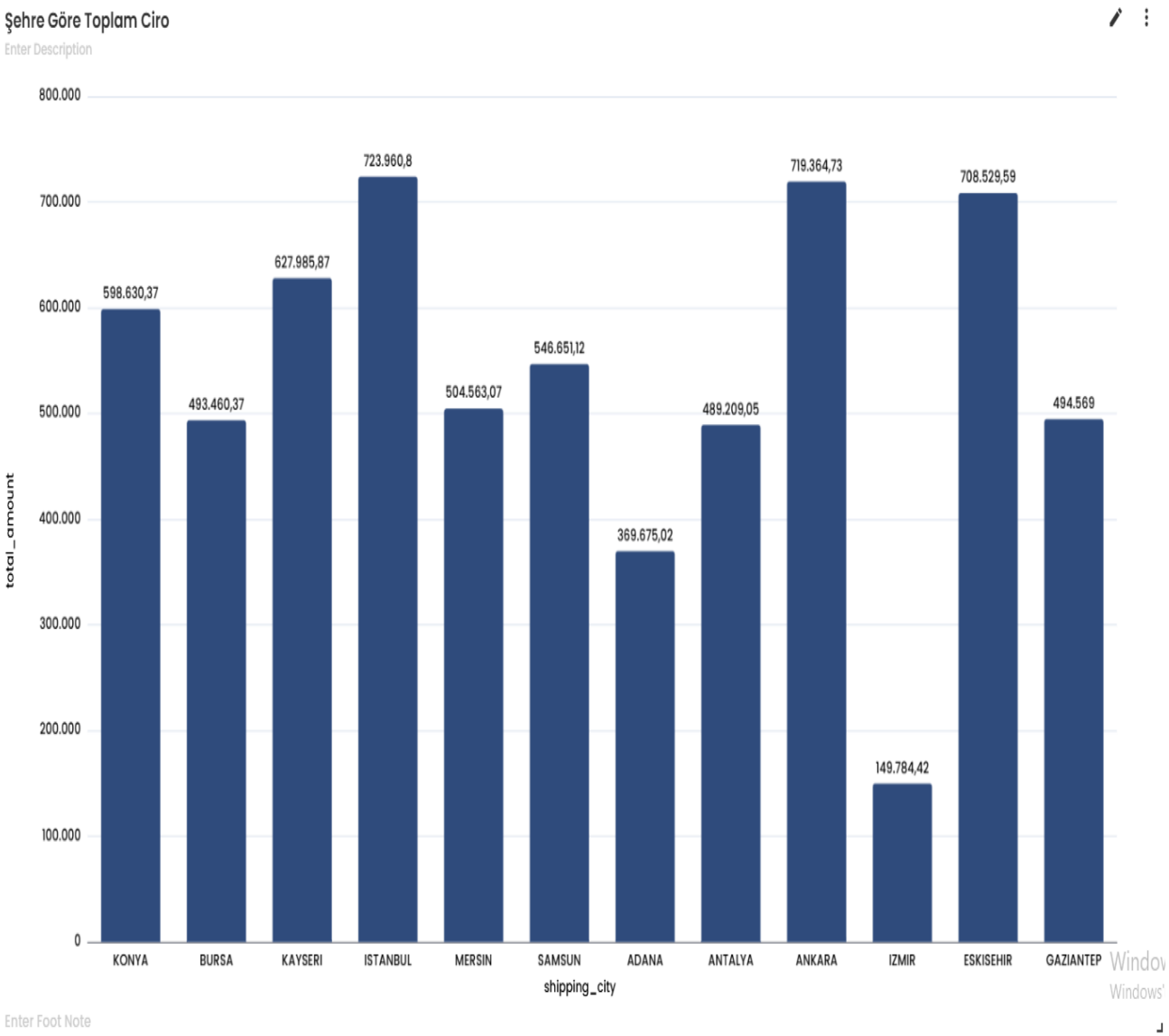
Enter Foot Note

Ödeme yöntemine Göre Dağılım

Enter Description

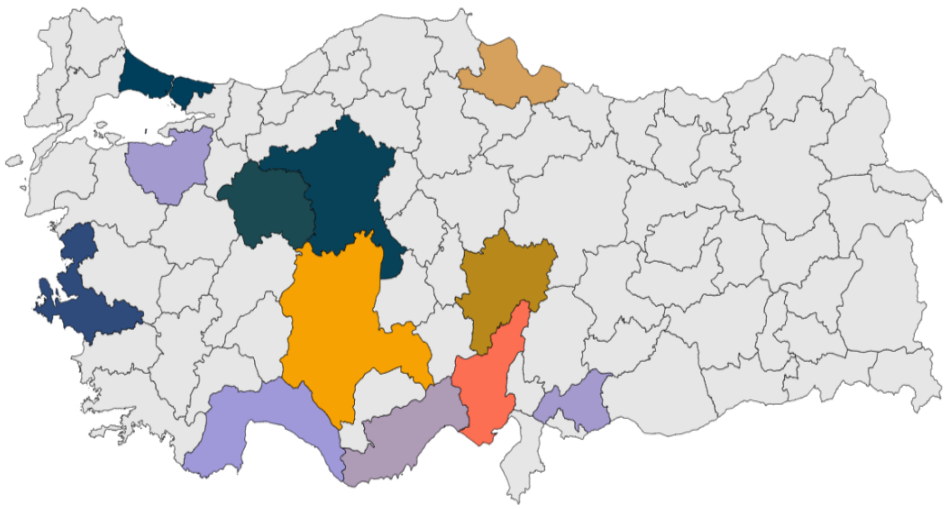


Enter Foot Note


Şehre Göre Toplam Ciro
Enter Description
800.000
700.000
600.000
500.000
400.000
300.000
200.000
100.000
0
total_amount
598.630,37
493.460,37
627.985,87
723.960,8
504.563,07
546.651,12
369.675,02
489.209,05
719.364,73
149.784,42
708.529,59
494.569
KONYA
BURSA
KAYSERI
ISTANBUL
MERSIN
SAMSUN
ADANA
ANTALYA
ANKARA
IZMIR
ESKISEHIR
GAZIANTEP
shipping_city
Enter Foot Note

Şehre Göre Ciro Haritası

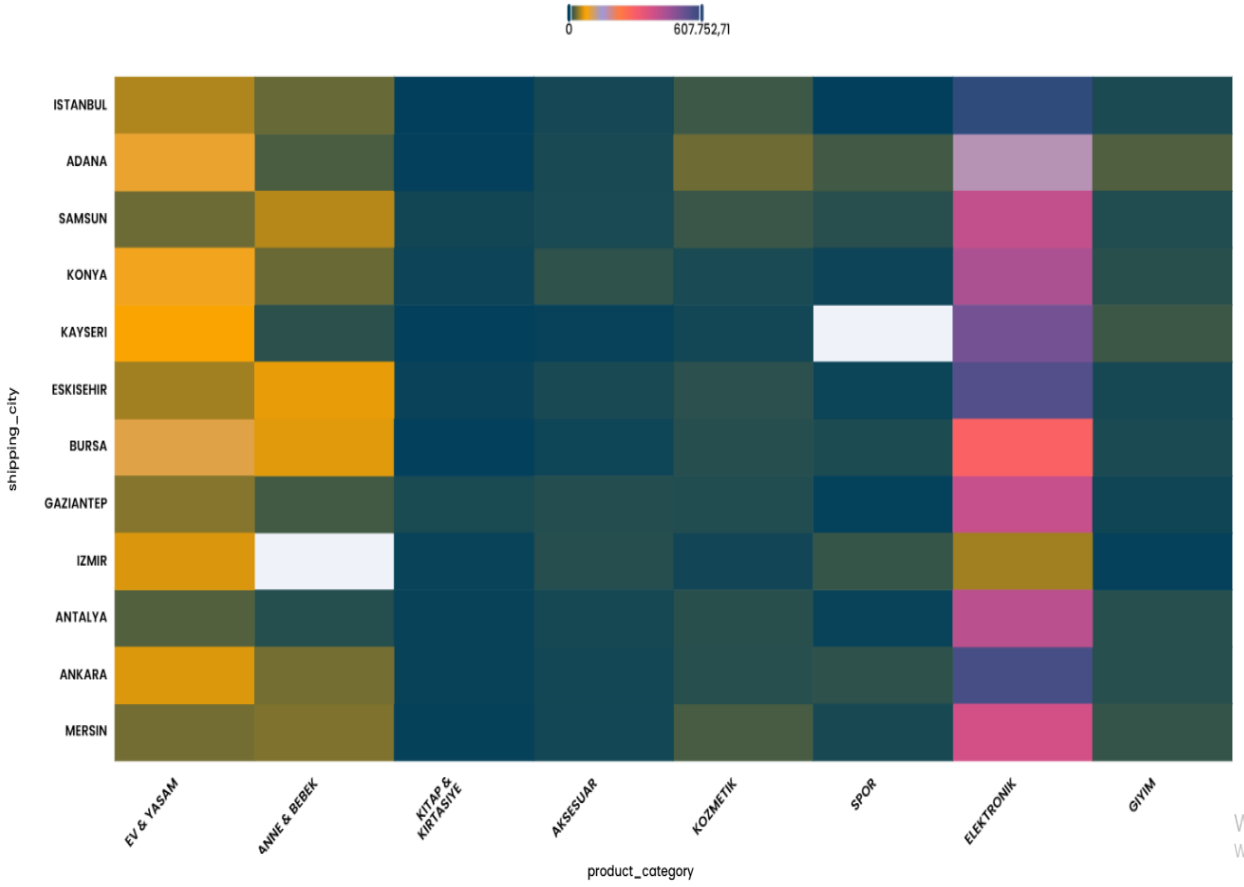
Enter Description



Enter Foot Note

Şehir ve Kategoriye Göre Satış Yoğunluğu

Enter Description



Enter Foot Note

Ürün Bazlı Satış Detayı

Enter Description

product_category	total_amount	quantity	product_name
ELEKTRONIK	2.376.994,42	95	Akıllı Telefon
ELEKTRONIK	2.200.073,23	120	Masaüstü Bilgisayar
ANNE & BEBEK	351.797,54	84	Bebek Arabası
EV & YASAM	311.710,89	117	Mutfak Robotu
EV & YASAM	222.454,15	118	Kahve Makinesi
EV & YASAM	180.914,42	85	Ofis Sandalyesi
ELEKTRONIK	106.441,58	75	Akıllı Saat
KOZMETIK	85.012,07	76	Parfüm
GIYIM	81.499,68	83	Erkek Ceket
KOZMETIK	80.185,85	112	Cilt Bakım Seti
	Total: 6.426.383,43	**Total: 1.731**	

Sonuç ve Değerlendirme:

Bu proje, bir yöneticinin tek cümlelik sorusunun ("Ben nerede ne kadar kazanıyorum? Hangi ürünlerden çok kazanıyorum?") arkasında aslında ne kadar uzun bir yolun bulunduğunu gösterdi. Veri, doğduğu anda hızlı ama kontrolsüz bir şekilde kaydedildi; analiz zamanı geldiğinde önce çıkarıldı (Extract), sonra Türkçe karakter sorunları, esnek olmayan tarih formatları, tekrar eden ürün kayıtları ve nihayetinde ciddi bir mimari yarış durumu (race condition) gibi art arda gelen sorunlarla baş edilerek dönüştürüldü (Transform), doğrulanmış bir yıldız şemasına yüklendi (Load) ve son olarak, kimsenin SQL bilmesine gerek kalmadan okuyabileceği bir dashboard'a dönüştürüldü (BI).

Bu yolculuğun sonunda, en baştaki soruya artık somut verilerle cevap verilebiliyor:
Şirket 2025 yılında toplam 6.426.383,43 TL ciro elde etti; 500 siparişin %85,2'si başarıyla tamamlandı.
Kazancın büyük kısmı Elektronik kategorisinden, özellikle Akıllı Telefon ve Masaüstü Bilgisayar ürünlerinden geliyor.
Coğrafi olarak İstanbul, Ankara ve Eskişehir en güçlü pazarlar; İzmir ise beklenenin belirgin şekilde altında bir performans gösteriyor.
Ödemelerde Havale/EFT ve Dijital Cüzdan, kredi kartı ve kapıda ödemenin önüne geçmiş durumda.

Bu bulguların hiçbiri, projenin başındaki ham veriden doğrudan çıkarılamazdı; çünkü o veri aynı şehri beş farklı şekilde yazıyor, aynı kategoriyi büyük/küçük harfle bölüyor ve miktar alanını bazen yazıyla, bazen negatif sayıyla tutuyordu. Bu bulguların güvenilir olmasının nedeni, verinin geçtiği Extract-Transform-Load-BI zincirinde her adımın ayrıca doğrulanmış olmasıdır.
Sonuç olarak ortaya, yöneticinin sorusuna dört farklı boyuttan (ürün, şehir, zaman, ödeme yöntemi) cevap veren, sayıları birbirini doğrulayan ve güvenilir bir karar destek aracı çıktı.

Peki Neler Yapılabilir?

Dashboard'daki bulgular, sadece geçmişi özetlemekle kalmıyor; şirketin önümüzdeki dönemde alabileceği somut aksiyonlara da işaret ediyor. Aşağıda, verilerden yola çıkarak önerilebilecek birkaç tavsiye yer alıyor:

Şehir ve kategori ısı haritasında görüldüğü gibi doğu illerinden veri seti içinde hiç sipariş bulunmuyor. Bu illerde satışın düşük olmasının nedeni pazarlama ve marka bilinirliğinin yetersizliği olabilir; bu bölgelere yönelik hedefli PR ve reklam bütçesi ayrılarak marka görünürlüğü artırılabilir.

İzmir'de hem toplam ciro (149.784,42 TL ile en düşük şehir) düşük hem de Anne & Bebek kategorisindeki satış sıfır olarak görünüyor. Sistemde adresi İzmir olarak kayıtlı müşterilere, özellikle bu kategoride özel indirim kampanyaları sunularak bu bölgedeki durgunluğun geçici bir talep eksikliğinden mi yoksa yapısal bir sorundan mı kaynaklandığı test edilebilir.

Aylık ciro trendinde Aralık ayının (831.953,48 TL) yılın en yüksek cirolu ayı olması dikkat çekiyor. Eğer bu artışın nedeni yılbaşı alışverişi ve kampanyalarıysa, önümüzdeki yıl bu döneme özel ürünlerde stok seviyeleri önceden artırılarak olası bir stok tükenmesi riski önlenebilir.

Ürün bazlı satış detayında Akıllı Telefon ve Masaüstü Bilgisayar'ın toplam cironun büyük bölümünü tek başına oluşturması, şirketin gelirinin sadece iki ürüne bağımlı olduğunu gösteriyor. Bu yoğunlaşmanın getirdiği risk (örneğin bu iki üründe yaşanacak bir tedarik sorununun cirodaki büyük etkisi) göz önüne alınarak, orta segment ürünlerin (Ev & Yaşam, Kozmetik gibi) satışını artıracak farklı satış kampanyaları değerlendirilebilir.

Ödeme yöntemi dağılımında "Belirtilmemiş" kategorisinin 180.284,5 TL'lik bir dilimi oluşturması, sipariş formunda ödeme yöntemi alanının zorunlu hale getirilmesi gerektiğine işaret ediyor; bu basit bir form değişikliğiyle, gelecekteki veri setlerinde bu tür eksikliklerin önüne geçilebilir.